JUST AUTOS

JA Portal — Handover

How the portal is built, where it runs, every connection it has, and the operating procedures for every module.

Compiled 20 August 2026 · justautos.app · Internal — not for distribution outside Just Autos

JA Portal — Full Handover Document

How the portal is built, where it runs, every connection it has (and how to set up or change them), and operating procedures for every module.

Compiled 20 August 2026 from the live codebase. Supersedes `03_SYSTEM_OVERVIEW.md` / `05_INTEGRATIONS.md` (May 2026) where they conflict.

---

1. What the portal is

An internal management platform for **Just Autos**, covering two MYOB business entities:

- **JAWS** — Just Autos Wholesale (distribution arm, holds stock, ~14 distributors across Australia)
- **VPS** — Vehicle Performance Solutions (the workshop entity; runs on **Mechanics Desk**, which remains the system of record — see §7.2)

It is one Next.js application that contains: a **staff portal** (dashboards, workshop management, CRM, AP automation, calls coaching, reporting), a **distributor-facing B2B portal** (catalogue, checkout, orders, tune jobs, training) that went live in July 2026, a **supplier portal** (read-only stock wall), and a large fleet of **background automation** (27 Vercel crons, 16 GitHub Actions workflows, and several on-premise agents).

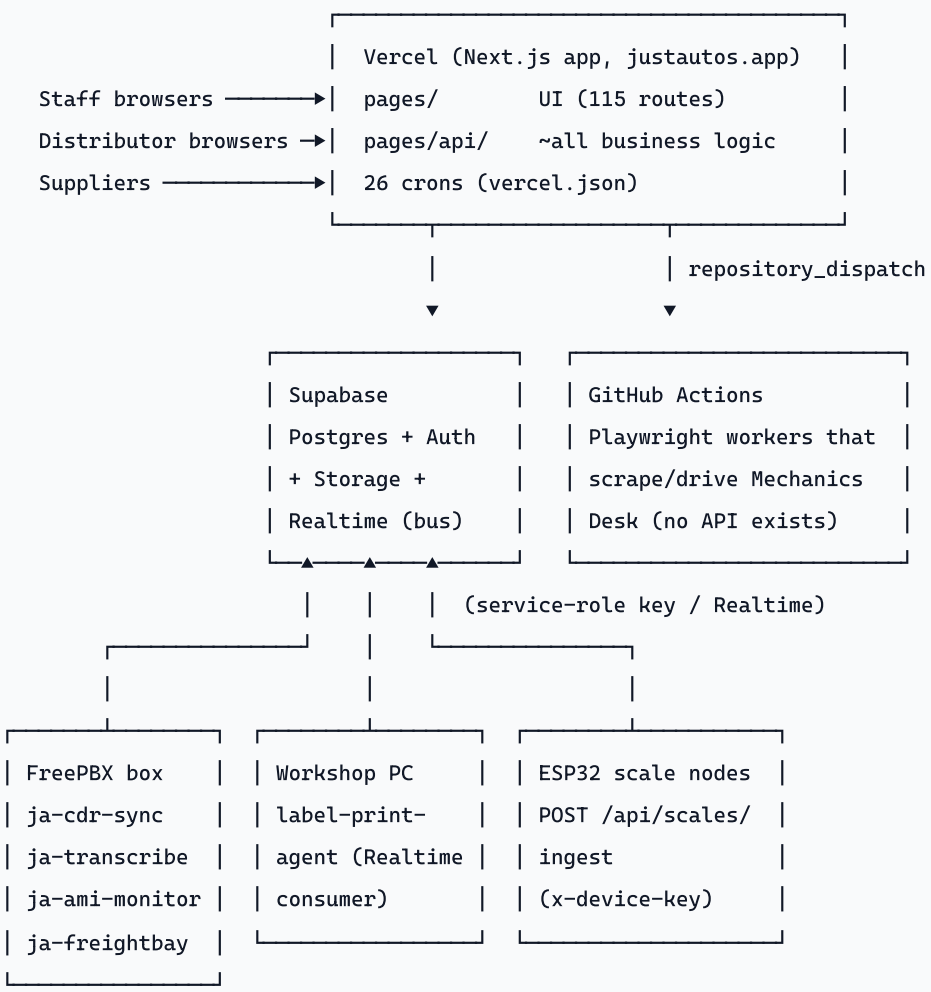
Production URL	`https://justautos.app`
Repo	`ChrisJustAutos/JA-Portal`, branch `main`
Hosting	Vercel (auto-deploys every push to `main`)
Database	Supabase project `qtiscbvhlvdvafwtdtcd` (`https://qtiscbvhlvdvafwtdtcd.supabase.co`)
Framework	Next.js 14.2.5, **pages router**, TypeScript, React 18
Key packages	`@supabase/supabase-js`, `@anthropic-ai/sdk` (mostly raw fetch is used), `exceljs` / `xlsx`, `@react-pdf/renderer`, `pdf-lib`, `leaflet`, `reactflow`, `sip.js`, `web-push`, `playwright` (dev, for GH Actions workers)

People

Person	Role
Chris Russell	Operations Manager — portal owner/direction
Nat	Accountant — chart of accounts, MYOB reconciliation sign-off
Matt Ashley	Technical — MYOB API, integrations
Matt H	Operations / Sales
Amanda	Accounts Payable — works in `/ap` daily
Laura	Director
Ryan	Receives the Monday 7am Weekly Sales Recap email

---

2. Architecture at a glance



External SaaS the Vercel app talks to directly: **MYOB AccountRight** (both entities, direct OAuth), **Xero** (migration target, foundation live), **Stripe** (B2B standalone account + JAWS read-only accounts), **Microsoft Graph** (mailbox webhooks + mail), **Resend** (email), **ClickSend** (SMS), **Slack** (bot + webhooks), **Monday.com**, **ActiveCampaign** (being replaced by CRM module), **MachShip** (freight), **Anthropic API** (18 LLM call sites), **Deepgram** (indirectly — runs on the PBX host), **Google Places** / **Nominatim** (geocoding).

Design conventions (must-follow)

- **API routes are `.ts`, never `.tsx`** — a `.tsx` in `pages/api/` crashes module load (generic 500 HTML).
- **Every `class X extends Error` MUST call `Object.setPrototypeOf(this, X.prototype)`** in its constructor. `tsconfig.json` targets **es5**, and TypeScript's downlevel of a class extending a built-in breaks the prototype chain, so `err instanceof X` is **always false** - silently. All seven custom error classes in the repo had this, and every one of them gates real error handling: MachShip 503/404/502, and the 409-plus-code responses for workshop invoices, payments, credit notes, stocktakes and POs. None of that handling had ever run; everything fell through to a generic 500. Found 2026-08-25 chasing why a dead MachShip consignment reported `getConsignment failed` instead of parking. Verified by compiling both shapes at `--target es5` (false vs true). Adding a new error class without this line reintroduces it, and nothing will fail loudly.

- **Shared UI kit is mandatory**: `lib/ui/theme` tokens (`T` — CSS variables; use `alpha()`, never alpha-suffixed tokens) + `components/ui` + Feedback hooks. No `alert()` / `confirm()` browser dialogs. The B2B portal has its own separate "Alloy" kit at `components/b2b/ui.tsx` (one accent, ≥12px type, 44px touch targets).
- **Page scroll uses `minHeight: '100vh'`, never `height: '100vh'` + `overflow: 'hidden'`** — a clipped full-height wrapper wrapped around a `flex: 1` scroll pane leaves the bottom of the page unreachable (on a phone the wrapper is taller than the visible viewport and the document itself can't scroll). Staff `/admin/b2b/*` pages put their content in `<main className="b2b-admin-main">`, which also carries the shared mobile CSS that makes wide tables scroll sideways. This bit the Training page (Aug 2026) and Tune Jobs (fixed 2026-08-24).
- **Server Supabase clients** are constructed inline per route with the service-role key (module-level memo `_sb`). There is deliberately no shared server client module — follow the local pattern.
- **Long work (>~30s) never runs on Vercel** — anything browser-based or slow goes to GitHub Actions (dispatch pattern) or is chunked.
- **PostgREST silently caps every response at 1000 rows** (`db-max-rows`), whatever `.limit()` asks for — no error, no flag, the surplus is just dropped off the end of the sort order. Any select whose result set can exceed 1000 rows must page through `selectAllRows()` (`lib/supabase-paged.ts`), ordering on a **unique** column. This silently broke the Monday Quotes & Jobs Map report (see §7.8).
- **PostgREST embeds on workshop tables must use the `!customer_id` hint** (e.g. `workshop_customers!customer_id(...)`) or the query 500s on ambiguous relationships.
- `b2b_distributor_users` **is not unique per email/auth_user_id** (multi-site memberships) — never `maybeSingle()` on it.
- An **update notifier** compares the client bundle against `/api/version` and shows a "new version — Reload" banner after each deploy.

---

3. Environments, deployment & dev workflow

Deploy

1. Commit to `main` and **push** (convention: always push immediately after committing — never leave local-only commits).
2. Vercel builds and deploys production automatically. There is **no staging branch**; PR preview URLs exist if wanted.
3. Function logs: Vercel dashboard → Deployments → deploy → Functions tab.

Error fingerprints: generic 500 HTML page = module-load crash (bad import / `.tsx` in api/ / syntax error); JSON error = handler crash; empty response/timeout = function `maxDuration` exceeded (per-route overrides live in `vercel.json` → `functions`).

Database migrations (SOP)

- Migrations live in `migrations/NNN_description.sql` (202 files, `002` – `202`; note `148` and `153` are each duplicated — sequence is a convention, not a key. Next number: **203**).
- There is **no migration runner**. The procedure is: write the SQL file in `migrations/`, apply it to the live DB via the **Supabase MCP** `apply_migration` tool (project `qtiscbvhlvdvafwtdtcd`) **before** pushing code that depends on it, then commit both.

- The repo file is the source-of-truth record; the MCP apply is what actually changes the DB.

Local dev

`npm run dev` with a `.env.local` carrying at minimum the Supabase URL/keys. Most features hit live external services, so local dev is mainly for UI work; be careful with anything that writes to MYOB/Stripe.

Remote/mobile ops

A workshop MSI laptop runs OpenSSH + Tailscale + Claude Code; Chris's phone (Tailscale + Termius) can SSH in from anywhere. This is also the **only interactive route to the FreePBX box** (Tailscale IP `100.82.97.46`).

---

4. Auth & access model

Four separate authentication schemes coexist. They never overlap.

4.1 Staff portal

- Supabase Auth session, cookie `ja-portal-access-token`; identity in `user_profiles`. Opt-in TOTP 2FA (server-side enforced).
- **Six roles** (`lib/permissions.ts` is the source of truth): `admin`, `manager`, `sales`, `workshop`, `accountant`, `viewer`. (`lib/auth.ts` has a stale 5-role type missing `workshop` — known drift.)
- Roles map to `view:*` / `edit:*` / `admin:*` permission strings via `ROLE_PERMISSIONS`. Pages gate with `requirePageAuth(ctx, 'permission')`; API routes with `withAuth(permission, handler)` / `requireAdmin`.
- Two per-user allowlists narrow further: `user_profiles.visible_tabs` (which nav tabs/apps a user sees) and `visible_report_tabs` (which Reports sub-tabs).
- **User management**: `/settings?tab=users` — invite (Supabase invite email → `/reset-password?welcome=1`), change role, deactivate, delete. Audit log at `/settings?tab=audit`.
- Session persistence: `SessionKeeper` silently re-syncs cookies from localStorage (never on auth pages) and login pages silently resume valid sessions; deep links use `?next=`.

4.2 Distributor (B2B) portal

- Separate cookies `ja-b2b-access-token` / `ja-b2b-refresh-token`; identity in `b2b_distributor_users`. Gate: `requireB2BPageAuth`.
- Login = email+password with optional TOTP; also magic-link and signed invite tokens (`/b2b/welcome?t=…` — scanner-proof: opening does nothing, only the human submitting the set-password form activates).
- Users are `owner` or `member` per distributor; one auth user can belong to **multiple distributors** (account switcher, `ja-b2b-dist` cookie).
- Admin **preview mode**: signed `b2b_preview` token from the admin side renders the portal as a distributor with all non-GET requests blocked.
- `checkoutEnabled` per distributor = browse-only kill switch.

4.3 Supplier portal

Separate again: `b2b_supplier_users`, `requireSupplierPageAuth`, single read-only page (`/b2b/supplier`).

4.4 Machine auth

- **Service tokens** (`lib/service-auth.ts`): SHA-256 hashed rows in `service_tokens` , presented as `X-Service-Token` header (deliberately not `Authorization` to avoid clashing with Supabase JWTs). Scoped (`stocktake:write` , `calls:monitor` , `ap:admin` , `reports:read` , `upload:job-report`). Used by GitHub Actions workers and the PBX AMI agent. Managed via `/api/admin/service-tokens` .
- **Signed capability tokens** in URLs for login-less pages: `/tune-jobs?token=` (distributor-scoped weekly reminder), `/order-action` (admin Book Freight email button).
- **Device keys**: ESP32 scale modules authenticate to `/api/scales/ingest` with `x-device-key` matched to `scale_devices` .
- **MCP tokens**: `jap_…` personal tokens (hashed in `mcp_tokens`) or OAuth 2.1 for the read-only Claude connector at `/api/mcp` .
- **Cron auth**: `Authorization: Bearer $CRON_SECRET` (some handlers also accept the `vercel-cron` user-agent, and some accept a logged-in staffer with the right permission for manual runs).

---

5. Connections & integrations

5.1 Where credentials live (read this first)

`lib/integration-config.ts` implements a **DB-first resolver**: `getIntegration(key)` = `integration_settings` DB row → env var of same name → `''` . Values cache in-process for 30s. **Always use this resolver, not `process.env` , for the managed keys.**

Integration	Stored where	Change without redeploy?
ClickSend, Resend, CRM intake token, **Xero app creds**, accounting provider switch	`integration_settings` DB (env fallback)	**Yes** — Settings → Connections → Integrations; live in ~30s
MYOB app creds, Graph, Monday, ActiveCampaign, Stripe, Slack, Anthropic, VAPID, Supabase, GitHub dispatch	Vercel env vars only	No — edit env + redeploy
MYOB / Xero **OAuth tokens**	`myob_connections` / `xero_connections` tables	Yes — re-run connect flow
MachShip token	`b2b_freight_carrier_connections.credentials` JSONB	Yes — Admin B2B → Settings
Service tokens / MCP tokens / scale device keys	`service_tokens` / `mcp_tokens` / `scale_devices`	Yes — respective admin UIs

Admin surfaces:

- **Settings → Connections** (`/settings?tab=connections`) — three sub-tabs: **Integrations** (edit DB-managed credentials, test SMS/email, rotate CRM intake token, Xero connect), **Health** (live status board), **MYOB Connection** (connect/select company files).
- `/admin/connections` — standalone auto-refreshing health page (⚠ no SSR auth gate — relies on API-level gating).

5.2 MYOB AccountRight (primary accounting — both entities)

- **Code**: `lib/myob.ts` (OAuth + `myobFetch`), `lib/myob-reporting.ts` (replaced CData 2026-07-14 — reporting **must fetch all 4 invoice types**), plus per-module writers (`ap-myob-bill`, `b2b-myob-invoice`, `workshop-myob-invoice`, `stripe-myob-sync`, …).
- **App creds (env only)**: `MYOB_CLIENT_ID`, `MYOB_CLIENT_SECRET`, `MYOB_REDIRECT_URI`, `MYOB_SCOPE`.
- **Tokens**: `myob_connections` table, one row per label `JAWS` / `VPS`. Access ~20 min (auto-refresh), refresh ~1 year but **must be exercised regularly** — a multi-week quiet spell can kill the connection.
- **Connect / re-auth SOP**: Settings → Connections → MYOB Connection → Connect (or `GET /api/myob/auth/connect?label=JAWS|VPS`) → consent → pick company file → if the file is in legacy auth mode, set company-file username/password. Test with `/api/myob/test/invoice?label=VPS`.
- **Gotchas** (hard-won, in code comments):
  - Two company-file auth modes — SSO (bearer only) vs legacy (`x-myobapi-cftoken`); header only sent when `company_file_username` is set.
  - **Never page with bare `$top` / `$skip`** — no stable ordering, rows get skipped. Follow `NextPageLink`, page size 400.
  - Every call is logged to `myob_api_log`; the insert is `await`ed in `finally` (fire-and-forget logs were killed on timeout).
  - `IsTaxInclusive` **matters for cent-exact invoices** — B2B invoices are pushed inc-GST with checkout-exact line pricing.
  - No P&L endpoint exists in AccountRight (the old CData P&L panels were retired).

5.3 Xero (migration target — foundation live 2026-08-05)

- **Code**: `lib/xero.ts`, `lib/accounting/xero-adapter.ts`; provider switch in `lib/accounting-provider.ts`.
- **App creds (DB-managed)**: `XERO_CLIENT_ID`, `XERO_CLIENT_SECRET`, `XERO_REDIRECT_URI` — editable in the Integrations tab.
- **Setup SOP**: create a Web App at developer.xero.com with the redirect URI shown on the card → paste ID/secret → Save → "Connect Xero" (`/api/xero/auth/start`) → one consent covers both orgs → map tenants to `VPS` / `JAWS` via `/api/xero/connections` PATCH → ping to verify.
- **Provider switch**: `accountingProvider(entity, module?)` resolves `ACCOUNTING_PROVIDER_{ENTITY}_{MODULE}` → `ACCOUNTING_PROVIDER_{ENTITY}` → default `'myob'`. Entity-wide keys are in the Integrations UI; per-module overrides (AP, STATEMENTS, LETTERS, WORKSHOP, INVENTORY, B2B, REPORTING, STRIPE, BANK) are env-only. After cutover MYOB stays connected read-only for history.
- **Critical gotcha**: **Xero refresh tokens are single-use and rotate on every refresh** — the new token must be persisted or the connection dies. Refresh is serialised in-process; a cross-instance race shows as `invalid_grant` and self-heals next run. Granular scopes only (bills live under `accounting.invoices`); every call needs the `Xero-tenant-id` header.

5.4 Stripe

- **B2B checkout (standalone account `acct_1TrRkZ…`, LIVE keys)**: `lib/stripe.ts` (raw fetch, no SDK). Env: `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`. Webhook `POST /api/b2b/stripe/webhook` (event `checkout.session.completed`; `bodyParser: false` is required for signature verification — do not remove). Idempotent; if the MYOB writeback fails it still 200s and parks the error on

`b2b_orders.myob_write_error` for manual retry from the admin order page. Payment methods live: **card (with surcharge), PayTo, BECS direct debit** (BECS settles later — a settle gate + `/api/cron/b2b-payment-check` confirm before fulfilment milestones).

- **JAWS reconciliation accounts (read-only)**: `lib/stripe-multi.ts`, env `STRIPE_SECRET_KEY_JAWS_JMACX`, `STRIPE_SECRET_KEY_JAWS_ET`, labels registered in `STRIPE_ACCOUNT_LABELS`. Used by `/stripe-myob` push tool and payout reconciliation.

5.5 Microsoft Graph (mail)

- **Auth**: app-only client credentials. Env: `GRAPH_TENANT_ID`, `GRAPH_CLIENT_ID`, `GRAPH_CLIENT_SECRET` (+ webhook URLs and ~15 mailbox-assignment vars — see `lib/microsoft-graph.ts` and §22 of the integrations inventory). Azure app needs **Mail.Read, Mail.ReadWrite, Mail.Send with admin consent** (Mail.Send 403 until granted — this bit us).
- **Webhooks**: `/api/webhooks/graph-mail` (Pipeline A — quote PDFs from rep mailboxes), `/api/webhooks/graph-jobreport-mail` (nightly MD WIP report).
- **Subscription lifecycle SOP**: subscriptions max out at ~70.5h. The cron `/api/cron/renew-graph-subscriptions` (every 6h) extends anything within 24h of expiry. **Recreating a dead/failed subscription is NOT automated** — re-run `POST /api/admin/setup-graph-subscriptions?key=$GRAPH_ADMIN_SETUP_SECRET` (idempotent).
- **Adding a rep mailbox** touches three places: `lib/agents.ts` (owner map), `MAILBOX_FOR_NAME` in `pages/api/cron/health-check.ts`, and re-running the setup endpoint.
- Inbox-driven pipelines that *poll* Graph rather than subscribe: AP inbox pull, AP auto-entry, AP statement watch, tune-jobs receipt scan, drop-ship confirmations, letter watch.

5.6 Email (Resend) & SMS (ClickSend)

- **Resend** (`lib/email.ts`): DB-managed keys `RESEND_API_KEY`, `RESEND_FROM`, `RESEND_CAMPAIGN_FROM`, `RESEND_REPLY_TO`, `RESEND_WEBHOOK_SECRET`. If unset, **silently falls back to Graph sendMail**. Delivery/open/bounce webhook: `/api/crm/resend-webhook?key=…`. Per-staff Reply-To comes from `user_profiles.reply_to_email`.
- **ClickSend** (`lib/clicksend.ts`): DB-managed `CLICKSEND_USERNAME`, `CLICKSEND_API_KEY`, `CLICKSEND_FROM`. Returns `clicksend_not_configured` harmlessly until set. Test buttons for both live in the Integrations tab.

5.7 Slack

- **Incoming webhooks** (one env URL per channel): `SLACK_WEBHOOK_URL` plus `SLACK_WEBHOOK_{JAWS_PAYMENTS,VPS_PAYMENTS,JAWS_PAYOUTS,AP_VPS}` and per-tenant B2B order webhook in `b2b_settings`.
- **Bot** (`lib/slack-bot/*`): env `SLACK_BOT_TOKEN`, `SLACK_SIGNING_SECRET`, `SLACK_ALLOWED_CHANNEL_IDS`, `SLACK_PARTS_ONLY_CHANNEL_IDS` (parts-only channels get exactly one stock-availability answer per top-level message, no thread replies, no clarifying questions), `SLACK_PARTS_CONTACT`. Single endpoint `POST /api/slack/ask` handles events, mentions, `/ask`, and interactive buttons; HMAC verified; acks in <3s then continues via `waitUntil`.
- The parts bot answers from `md_stock_cache`, refreshed every 30 min by the `md-stock-sync` GitHub Action. **"On cars" checker (Stocktake MD).** `md-oncar` → `scripts/pull-md-oncar.ts` → `collectPartsOnCars()` → `POST /api/workshop/oncar/ingest` (service token `stocktake:write`) → `md_oncar_runs` / `_items` / `_jobs` / `_job_items` (migration 205). `GET /api/workshop/oncar` serves the newest **done** run (so a refresh

in flight never blanks the screen) while reporting the newest run's state for the spinner; `POST /api/workshop/oncar/refresh` (`edit:stocktakes`) pre-creates the pending row then `repository_dispatch`es `oncar-pull`. Panel: `components/workshop/PartsOnCarsPanel.tsx` on `/stocktake`, with client-side CSV export of the filtered view (by-part, or by-car flattened to one row per part per car). Export is pure client-side — no route, no server render — unlike Pre Pick's PDF; add `lib/workshop/*-pdf` alongside it if a stylised sheet is ever wanted.

What qualifies, and why it was probed rather than assumed. A part counts only when it is on a **started** job — MD diary status `new`, whose date has arrived, whose invoice is **not finalized** (`invoice.finalized` is MD's stock-deduction gate), carrying tracked stock lines. Probed 2026-08-27:

- MD has exactly three job statuses: `new`, `preparing`, `finished`.
- `preparing` **is MD's forward forecast** — jobs booked ahead with parts prepped — and is EXCLUDED. It held 48 jobs / 404 units against a real figure of 17 jobs / 185.5 units, so counting it would more than triple the answer.
- **There is no usable jobs-list endpoint.** `/auto_workshop/{jobs,job_list,open_jobs,job_board,wip,kanban}` all 404 and `/jobs.json` 504s. Enumeration is a day-by-day diary sweep (`/auto_workshop/diary`), **backwards only** — which is also what makes "the booking date has arrived" automatic. Default lookback 365 days; the oldest qualifying job when probed was 209 days old.
- `on_hand - available_quantity == allocated_quantity` **exactly** (30/30 sampled). So MD's allocation is computable from the cached stock list — but it counts future bookings too, making it an **upper bound**, not this figure. Don't be tempted to swap the job walk for it.

The worker shares the `mechanicdesk-session` concurrency group and re-logs in on eviction: a truncated sweep would render as "nothing is on a car", the most misleading answer this screen can give. `days_failed` is surfaced in the UI for the same reason.

- **The cache is all-or-nothing.** MechanicDesk allows one session per employee account, so a human (or another worker) logging in mid-run evicts the scraper. `fetchAllStock` therefore re-logs in and retries the same page on a 401, and **throws** if it finishes short of the page count MD reported on page 1 — the run is marked `error` and the previous cache is left alone. Before this (fixed 2026-08-27) a mid-run eviction silently truncated the catalogue and wrote the partial set over a good one: on 2026-08-26 the cache dropped from 856 items to 270 and reported success, leaving the front counter unable to find two-thirds of the catalogue. If the parts bot starts saying "not found" for stock you know exists, check `md_stock_sync_runs.item_count` against recent runs first.

What we post where. Slack is the business's alerting surface — most automation reports into it rather than email, so a dead webhook is a silent failure. The channels built so far:

Channel	What lands there	Source
`#jaws-orders`	B2B distributor orders, drop-ship PO confirmations and ETAs	`lib/b2b-order-pipeline.ts`, drop-ship confirm cron
`#jaws-payments` / `#vps-payments`	Payments received, per entity	`SLACK_WEBHOOK_{JAWS,VPS}_PAYMENTS`

Channel	What lands there	Source
`#jaws-invoices` / `#vps-invoices`	Invoice events per entity	webhook
`#sales-coaching`	The **weekly** team coaching summary only (Mon 07:00). Per-call cards are off, and the daily coaching recap moved into the 5:15pm `#sales-updates` post on 2026-08-31	PBX `slack-poster.js`
`#customer-feedback-negative`	Calls flagged as a poor customer experience, with the fault theme. Matt's standing ask is the top-10 recurring faults off this channel. **The automation that posts here is currently switched OFF** (`CALL_CONCERNS_ENABLED`).	calls analysis
`#customer-feedback-positive`	The same in reverse — calls that went well	calls analysis
`#parts-room`	Parts Office camera motion, nights and weekends only	`ja-partsroom` on the PBX box
Freight bay	Intrusion bursts from the freight-bay NVR (snapshot sequence)	`ja-freightbay` on the PBX box — Slack-only since 2026-07-15
AP channel (`SLACK_WEBHOOK_AP_VPS`)	Supplier-invoice exceptions and suspected double-ups (♻)	AP auto-entry
`#ja-portal-queries`	Ask-the-portal bot conversations	`lib/slack-bot/*`

⚠ Webhook URLs are per-channel env vars. If a channel goes quiet, check the env var before assuming the feature broke — a rotated or deleted webhook fails silently.

5.8 Monday.com & ActiveCampaign (legacy CRM pair — being replaced by the CRM module)

- **Monday** (`lib/monday-followup.ts` is the shared GraphQL client): env `MONDAY_API_TOKEN` , `MONDAY_BUTTON_SECRET` . Five rep quote boards (Dom 5025942308, Kaleb 5025942316, Graham 5026840169, James 5025942292, Tyronne 5025942288). Pipeline A is match-only (never creates items). Button callback: `/api/monday/fetch-call-notes?key=…` .
- **Quote-channel automations (rebuilt 2026-08-19/20)**: a 3/7/14-day follow-up cadence driven by an `FU Stage` column (1→2→3; "Follow Up Done" advances the stage, resets Contact Attempts, hides the item in Quote - Pending and re-surfaces it when the date arrives). RLMNA counts attempts: **5 tries in Quote - Lead RLMNA → Quote Not Issued**, **3 tries in Quote - Follow up RLMNA → Quote Lost**. After the third follow-up the owner is notified that the decision must be Won or Lost.
  - **Two landmines, both fixed 2026-08-20, worth knowing if the boards misbehave again.** (1) Graham's legacy `RLMNA - Step 1` had no group condition and his `Follow up RLMNA - Step 1` incremented the counter twice, so one RLMNA click added +3 attempts and lost the quote on the first click; rebuilt as new-builder workflows 1940255686 / 1940255690 with the move ordered *before* the increment. (2) The Follow-

Up-Done handlers are gated on `FU Stage is exactly 1/2/3`, so a **blank** FU Stage makes the click do nothing at all, silently — 132 items (mostly the On Hold cohort the migration skipped) were backfilled to stage 1.

- **MCP cannot delete or deactivate Monday automations** (`USER_UNAUTHORIZED`, even for the account owner — the connector lacks the scope). The workaround is always: create a corrected version, add the bad one to a manual delete list. `create_automation` is also **nondeterministic** — always re-dump with `list_automations` and check the block config; phrase status triggers as "changes to X (from any previous status)" or it will set the from-status to the same label and the workflow can never fire.
- **End of cadence (2026-08-20).** After the third follow-up the item deliberately does NOT move — it stays in Quote - Follow Up awaiting a Won/Lost decision. Two changes made that legible: a **"Decision Required"** status label the stage-3 handler now sets (workflows 1940257758 Ka / 1940257760 Ty / 1940257765 Ja / 1940257773 Do / 1940257784 Gr), so the row visibly differs from a transient "Follow Up Done"; and the **Owner** column is now stamped by Pipeline A (`REP_BOARDS[].mondayUserId`). ⚠ **Owner was blank on every item on every board**, so the existing "notify the Owner" automation had been reporting success while notifying nobody — ten of Kaleb's quotes sat at the decision point from mid-June. Historical items were backfilled 2026-08-20 out of hours: **704 active quotes** across the five boards (Tyronne 84, James 188, Kaleb 294, Dom 169, Graham ~111) — scoped to Quote - Pending, Follow Up, Follow up RLMNA and On Hold only. Won, Lost, Not issued and the pre-quote Lead groups were deliberately left blank. Verified: zero blank Owners remain in those 20 groups.
- ⚠ **Never bulk-write to these boards while a rep is working in one.** The item state can change between your read and your write — this bit on 2026-08-20: ten items were read at stage 3, Kaleb reset them to stage 1 and re-ran them seconds later, and the write landed on the new state and had to be undone.

- **ActiveCampaign** (`lib/activecampaign.ts`): env `ACTIVECAMPAIGN_API_URL`, `ACTIVECAMPAIGN_API_KEY`, owner map etc. **Landmine documented in code: `filters[phone]` is a partial match and matches everything on an empty query** — every match must be re-verified by exact digit comparison (this mis-attributed a contact to 26 customers in production once).

5.9 MachShip (B2B freight)

- **Code**: `lib/b2b-machship.ts` + `lib/b2b-freight-carriers.ts` (provider registry — Shippit/Starshipit/AusPost/Sendle slots exist but only MachShip is implemented).
- **Credential**: NOT env — `b2b_freight_carrier_connections` row `provider='machship'`, JSONB `api_token`, header `token:` (not Bearer). Change it in Admin B2B → Settings; effective next call.
- Base `https://live.machship.com` (no sandbox host — test vs live is a property of the token's MachShip user). **Manifesting needs `companyId`, obtained via a consignment GET.** Booking runs with `maxDuration: 120`; `/api/cron/b2b-freight-poll` refreshes status/ETA every 30 min.
- **Stale consignment ids self-heal (2026-08-25).** Deleting and re-creating a consignment in MachShip issues a NEW internal id, so our stored `machship_consignment_id` 404s forever while the shipment moves on happily under the same carrier tracking number. On a 404 `refreshOrderFreight` now re-resolves via `returnConsignmentsByCarrierConsignmentId` (tracking number — the durable anchor, it's on the label), then `returnConsignmentsByReference1` (our `"<order number> / <customer PO>"` customerReference), adopts the new id and carries on. It only accepts an unambiguous single match — attaching an order to the wrong shipment is far worse than staying parked. There is deliberately **no** lookup by our stored `MS…` consignment number: MachShip publishes no endpoint for it and a re-created consignment gets a new one anyway.

- MachShip documents those two paths but **not their request bodies**, and its Swagger is behind auth, so `lookupConsignments()` negotiates the shape — bare array, then `{ <named>: [...] }`, then `{ values: [...] }` — keeping whichever MachShip accepts, and returns `[]` (not a throw) if none do. Worst case is the previous behaviour, never a regression. If MachShip ever documents the real shape, collapse this to the one that works. Both endpoints cap at 10 values per request.
- **MachShip finishes consignments as `Complete`, not only `Delivered`.** `TERMINAL_DELIVERED` listed only `delivered`, so B2B-2026-000047 sat on `shipped` with a null `delivered_at` for six days after TNT delivered it. Now `{delivered, complete, completed}`.
- **`isManifested` can't be read off our own manifest id.** A consignment manifested outside the portal leaves `machship_manifest_id` null forever while the carrier reports the freight moving and finishing — so the order kept offering **Ship Now**, which would re-manifest it and raise the tax invoice a second time. Both the client gate (`pages/admin/b2b/orders/[id].tsx`) and the server gate (`isManifested()` in `lib/b2b-ship-now.ts`) now treat any status outside `{'', unmanifested, pending, pending_manifest}` as despatched. **These two must change together** — the client decides whether the button appears, the server decides whether it works.
- Consequently the poller no longer excludes `consignment_missing` forever: a second, smaller pass (`RETRY_MISSING_BATCH` 5) retries parked orders every `RETRY_MISSING_HOURS` (6), in its own batch so a backlog of genuinely dead ones can't crowd out live orders.

5.10 Phones: FreePBX + Deepgram + live monitoring

The portal never calls Asterisk or Deepgram directly — an on-PBX host (CentOS 7, Tailscale `100.82.97.46`) runs four workers and **Supabase is the bus**:

PBX worker	What it does
`ja-cdr-sync` (`sync.js`)	CDR → Supabase `calls` every 5 min (service-role key). Includes park-pickup CDR fix (2026-08-07), the 6-hour late-arrival lookback (2026-08-21) and extension-based outbound detection (2026-08-31) — all below.
`ja-transcribe` (`transcribe.js`)	New calls → Deepgram (`nova-2-phonecall`, `en-AU`) → `call_transcripts`. Deepgram key lives on the PBX host.
`ja-ami-monitor`	Live channel snapshots → `POST /api/calls/live/agent/snapshot` (~2s, `X-Service-Token` scope `calls:monitor`); drains `call_monitor_events` for Listen/Whisper/Barge and click-to-dial originate.
`ja-freightbay` / `ja-partsroom`	Hikvision NVR intrusion events → Slack snapshot bursts + Yealink ring. **Node 16 only** (glibc). Doesn't touch the portal API.

Call coaching and notes — what this actually gives the business. This is the biggest piece of bespoke work on the phone system, so it is worth stating plainly what it produces:

1. **Every call is recorded and transcribed** — `ja-cdr-sync` lands the CDR, `ja-transcribe` sends the audio to Deepgram (`nova-2-phonecall`, `en-AU`) and stores the transcript. Nothing is manual.
2. **Every call is scored against a rubric for its type** — a sales enquiry, a service booking, a parts call and a pass-off are judged on different things, so the rubrics are per call type (`call_type_rubrics`, editable at Settings → Call Coaching).

3. **Coaching is attributed to the advisor who actually handled it** - from their own extension since 2026-08-31, with the transcript overriding it when someone answers at another desk — that is what makes the leaderboard trustworthy when calls get transferred or picked up from park.
4. **Coaching reaches Slack once a day, inside the 5:15pm `#sales-updates` post** (§7.3), not as a card per call. A **weekly team summary still posts to `#sales-coaching` Monday 07:00**.
5. **Call notes flow back to the quote boards** — the "Fetch Call Notes" button on a Monday item calls `/api/monday/fetch-call-notes`, which pulls that customer's call history and notes onto the item, so a rep picking up a follow-up can see what was last said without hunting for the recording.
6. **Sentiment / objections / conversion** are surfaced on `/calls` as tabs, and calls scoring below 40 are flagged for attention.

Outbound is recognised by the EXTENSION, not the caller ID (2026-08-31). `sync.js` classified a call as outbound with:

```
const isOutbound = first.src === BUSINESS_NUMBER && KNOWN_EXTENSIONS.has(first.cnum);
```

An extension whose outbound caller ID has no **number** set leaves `src` empty in the CDR. **Extension 4001 is configured that way** (`clid` is `"Graham" <>`, `src` empty, `outbound_cnum` empty), so every one of its outbound calls failed that test, matched neither branch, and was discarded by the `return null` immediately below. **231 calls disappeared in 30 days** — every missing outbound call in that window was 4001's, and none of it surfaced until Chris noticed his own outbound calls to one number were absent. 4001 was never the problem in the extension list; it is in `KNOWN_EXTENSIONS`.

The test now keys off the extension that placed the call, which does not depend on PBX config:

```
const isOutbound = KNOWN_EXTENSIONS.has(first.cnum)
  && !KNOWN_EXTENSIONS.has(String(first.dst || ''))
  && String(first.dst || '').replace(/\D/g, '').length >= 6
  && (first.src === BUSINESS_NUMBER || !first.src);
```

- **`>= 6` digits, not 8.** Australian 13-numbers are six long and are dialled from here (`131008`); an 8-digit floor silently dropped three of them. Caught by validating the rule against 30 days of CDR before shipping rather than after.
- **Excluding destinations that are themselves extensions** is what keeps internal ext-to-ext calls out — the job `src === BUSINESS_NUMBER` used to do implicitly.
- Validated across 30 days: **+231 recovered, 0 lost, 0 internal calls captured.**

Reconciliation method, for next time. SSH to the PBX (Tailscale `100.82.97.46`), take the first CDR row per `linkedid` over the window, apply the same two predicates in a script, and compare the buckets against `public.calls`. Doing that produced: inbound 1,980 in the CDR vs 1,982 in the portal (window edges); outbound 2,592 vs 2,592 **exactly**; and a 669-row dropped bucket that was 411 internal calls, 27 setup rows and 231 genuine outbound. The CDR's raw "inbound" count is ~18,500 and is **not** a useful figure — most of it is SIP scanners hitting the trunk with 3-5 digit source numbers; only `did = 0754760066` rows are real calls.

Backfilling recovered calls. `BACKFILL_FROM` / `BACKFILL_TO` (MySQL datetime) run a bounded re-read that deliberately does **not** move the watermark and skips the recording-upload phase. That is safe because phase 2 is independent: normal runs upload any call that has a `recording_file` and no `recording_url`, so backfilled

rows collect their audio on subsequent runs at `RECORDING_UPLOAD_PER_RUN` (5) a run — about four hours for 231 calls. Recordings on the box go back to 2023, so audio is available far beyond any realistic backfill.

Still open: ext 4001 has no outbound caller-ID number. Left alone deliberately (Chris, 2026-08-31) — setting it changes what customers see on Graham's outgoing calls, which is a business decision, not a bug fix. The sync no longer depends on it.

The CDR lookback — why it is 6 hours and must stay generous. Asterisk stamps a CDR row with the call's **start** time but does not write the row until the channel hangs up. A long call therefore arrives *behind* the sync watermark. `sync.js` reads `WHERE calldate > (watermark − CDR_LOOKBACK_MINUTES)`; while that lookback was 30 minutes, **every call longer than about 30 minutes was silently lost or truncated** — the row appeared in MySQL after the watermark had already moved past its start time, so no run ever saw it. Symptom: the portal held no call longer than 38:37 in seven weeks, and a 61:20 call answered by Dom on 204 (2026-08-20 16:16, caller 0428673886) was absent entirely. Fixed 2026-08-21 by raising the lookback to **360 minutes** (`CDR_LOOKBACK_MINUTES`, env-overridable). Re-reading is cheap (~26 rows a run) and safe — the upsert is `on_conflict=linkedid` and the payload carries only CDR-derived columns, so transcripts, recording URLs and coaching analysis on existing rows are never overwritten. **Do not lower it below the longest call the phones can carry.**

Backfilling history. `BACKFILL_FROM` + `BACKFILL_TO` (PBX local time) make a hand-run read exactly that window, leave `sync_state` untouched and skip the recording-upload phase, so it never disturbs the 5-minute timer:

```
BACKFILL_FROM='2026-08-20T16:11:00' BACKFILL_TO='2026-08-20T16:21:00' \
  /usr/local/bin/node20 /opt/ja-cdr-sync/sync.js
```

Keep the window tight. A wide one re-upserts every call inside it, and `trg_rescue_missed_call` fires on `UPDATE OF disposition` — which deletes *unread* "Missed call" notifications for the same caller within ±4 hours of each answered call. There are routinely a couple of thousand unread ones, so a mass re-upsert would quietly clear a slice of them. (`trg_notify_missed_call` is safe: it only fires for calls under 2 hours old.) Note the box's default `node` is v8 and cannot parse the file — the service runs `/usr/local/bin/node20`.

Recordings are transcoded to MP3 before upload. Asterisk records 8 kHz mono WAV at roughly 1 MB per minute, so length translated straight into megabytes and long calls hit the storage ceiling — a 61-minute call was 58 MB and was rejected outright, leaving it with no audio, no transcript and no coaching card. Since 2026-08-21 `sync.js` runs the file through `ffmpeg` (`-ac 1 -ar 16000 -c:a libmp3lame -b:a 32k`) and uploads the MP3 instead: **~75% smaller** (58 MB → 14.7 MB), which puts even a multi-hour call comfortably inside the bucket's 100 MB limit. Knobs: `RECORDING_TRANSCODE=0` disables it, `RECORDING_MP3_BITRATE` (default `32k`), `FFMPEG_BIN`, `TRANSCODE_TMP_DIR`, `TRANSCODE_TIMEOUT_MS`.

Details that matter if you touch this:

- **The WAV on disk is never modified** — Asterisk owns it and it stays the source of truth. Only the uploaded copy is compressed, and a failed or empty transcode falls back to uploading the original WAV, so this can never cost a recording.
- **`recording_file` stays `.wav` while `recording_url` becomes `.mp3`.** They deliberately differ. Anything deciding a content type or a download name must read `recording_url` — that is the object that exists. `transcribe.js` derives the Deepgram content type from it, and both `lib/calls-weekly-report.ts` and `/api/calls/[id]/recording-url` were corrected to do the same.

- Old `.wav` objects stay exactly as they are and still play; the portal's `<audio>` element and Deepgram both handle either format.
- `transcribe.js` falls back to inserting the transcript **without** `raw_response` if the full insert fails. That column holds the entire Deepgram document with word-level timings (~940 kB on a 61-minute call) and nothing reads it, so a statement timeout on the payload must not cost the transcript — which is exactly what happened on the first 61-minute call before the fallback existed.

Analysis runs on the PBX (`/opt/ja-cdr-sync/analyse.js` + `slack-poster.js`); the portal's `/api/cron/calls-analyse` equivalent is **gated off by default** (`CALLS_ANALYSIS_ENABLED`) and must stay off while the PBX loop runs — both running means double-scoring and double-posting. Browser softphone (SIP.js over WSS) config is env `NEXT_PUBLIC_FREEPBX_WSS_URL` etc.; TURN needed on mobile networks.

5.11 Anthropic / Claude API

Env `ANTHROPIC_API_KEY` ; raw fetch to `/v1/messages` . 18 call sites: AP extraction & statement parsing, quote PDF extraction, call analysis/coaching/concerns/weekly report, follow-up summaries, workshop-map weekly narrative, B2B drop-ship confirmation reading, training-course generation, tune-job receipt extraction, report narratives, sales-recap flags, Slack bot, portal chat. Most models overridable per-feature via env (`*_MODEL` vars).

5.12 Everything else

- **Web Push**: `NEXT_PUBLIC_VAPID_PUBLIC_KEY` / `VAPID_PRIVATE_KEY` (watch for whitespace in env values — known gotcha); dormant no-op when unset.
- **MCP connector**: portal is a read-only MCP server at `/api/mcp` (personal `jap_` tokens from Settings, or OAuth 2.1 with dynamic client registration).
- **Google Places** (`NEXT_PUBLIC_GOOGLE_PLACES_API_KEY` , client-exposed — must be referrer-restricted) for workshop address autocomplete; **Nominatim** (keyless, throttled ~1 rps) geocodes tune-job addresses; an offline AU postcode dataset powers the workshop map.
- **GitHub dispatch**: `GH_DISPATCH_TOKEN` (`actions:write`) lets portal routes fire `repository_dispatch` events to run MD workers on demand.
- **Website lead intake**: `POST /api/crm/intake` guarded by `x-crm-token` = DB-managed `CRM_INTAKE_TOKEN` (rotate from the Integrations tab).
- **CData MCP**: decommissioned 2026-07-14. `lib/cdata.ts` is dead code; health check hardcoded green/deprecated.
- **Base URL sprawl**: seven overlapping env vars (`NEXT_PUBLIC_APP_URL` , `NEXT_PUBLIC_SITE_URL` , `NEXT_PUBLIC_BASE_URL` , `NEXT_PUBLIC_PORTAL_URL` , `PORTAL_PUBLIC_URL` , `APP_BASE_URL` , `JA_PORTAL_BASE_URL` , `B2B_PUBLIC_BASE_URL`) — keep them consistent; mismatches silently break OAuth redirects and email links.

---

6. Scheduled automation

6.1 Vercel crons (27 — `vercel.json` ; schedules are UTC; Brisbane = UTC+10)

Cron	Brisbane time	Purpose
`/api/distributors/refresh-cache`	02:00 daily	Warm `distributors_cache` (FY figures + last 13 months)

Cron	Brisbane time	Purpose
`/api/cron/sync-followups`	every 5 min	Call follow-ups → Claude summary → AC contact + Monday push (`FOLLOWUP_SYNC_ENABLED` kill switch)
`/api/cron/calls-analyse`	every 5 min	Portal-side call coaching sweep — **off by default** (`CALLS_ANALYSIS_ENABLED`); PBX box currently does this
`/api/cron/calls-weekly-report`	Mon 07:00	Weekly coaching narrative → `#sales-coaching`
`/api/cron/workshop-map-weekly`	Mon 07:10	Quotes/jobs geography report → email Matt (cc Ryan, Chris). Its md_quotes/md_invoices reads are PAGED (`selectAllRows`) — a plain select hits the 1000-row cap and under-reports
`/api/cron/renew-graph-subscriptions`	every 6 h	Extend Graph mailbox subscriptions expiring <24h
`/api/cron/health-check`	every 5 min	The 21 integration health checks → `integration_health`
`/api/cron/b2b-freight-poll`	every 30 min	MachShip status/ETA poll on open consignments
`/api/cron/workshop-reminders`	every 15 min	Queue + send service/rego reminder SMS (respects `sms_enabled`)
`/api/cron/notifications-sweep`	every 15 min	Bell notifications for Monday-side events (new leads, new to-dos)
`/api/cron/b2b-payment-check`	every 6 h	Two passes: confirm BECS settlement in MYOB → mark `payment_settled_at` ; and receipt any settled order that has no MYOB payment against it
`/api/cron/b2b-reminders`	every 3 h	Abandoned carts (24 h + 72 h), unfinished checkouts (once at 24 h), and paid orders we haven't shipped (2 d, escalating at 5 d)
`/api/cron/sales-update`	hourly :15, 17:15–21:15 Brisbane	**The single end-of-day post** to #sales-updates: sales figures (the day Mon–Thu, the week on Friday) **plus the call-coaching recap**. Marker-guarded so it posts once. `maxDuration` 300 — it makes the Anthropic theme call
`/api/cron/crm-automations`	every 5 min	CRM automation flow engine
`/api/cron/task-automations`	every 5 min	Tasks automation flow engine
`/api/cron/crm-campaigns`	every 5 min	Campaign scheduler + Resend queue drain + call linkage
`/api/cron/ap-statement-watch`	every 10 min	Supplier statements → reconcile vs MYOB → digest email (report-only)
`/api/cron/ap-auto-entry`	every 15 min	VPS inbox → fact-check → auto-post clean invoices to MYOB (`AP_AUTO_ENTRY_ENABLED`)

Cron	Brisbane time	Purpose
`/api/cron/overnight-leads-snapshot` (+morning variant)	every 30 min / 5 min 06–08	Snapshot Monday lead groups for the sales recap
`/api/cron/letter-watch`	hourly	New finalised MYOB VPS invoices → thank-you letter + envelope print jobs (deposit-only invoices vetoed)
`/api/cron/agents`	every 15 min	Monitoring agents framework (`lib/agent-framework`)
`/api/cron/tune-jobs`	hourly :20	Scan inbox for Stripe tune receipts; weekly distributor reminders + recap (Fri 08:00 Brisbane, retried through Sunday); escalations
`/api/cron/b2b-dropship-confirm`	every 15 min	Supplier confirmation emails → full drop-ship receiving flow
`/api/cron/mgmt-dashboard-warm`	05:30 daily	Pre-compute Management Dashboard MYOB bundles
`/api/cron/leave-decisions`	every 15 min	Leave applications decided on the Monday board → email the applicant (approved *and* denied), cc/reply-to HR. Only items on the application path count — see §7.17. `LEAVE_EMAILS_ENABLED=false` switches it off
`/api/cron/jaws-stock-eom`	07:30 on the **2nd**	JAWS month-end stock report for the month just ended → snapshot (migration 199) + email to `JAWS_EOM_EMAIL_TO` . Cron can't express "last day of month"; running into the new month also guarantees the month's invoices are all in. See §7.18

⚠ Two handlers exist but are **not scheduled** (headers claim otherwise): `bank-payments-slack.ts` (7am payment digests) and `slack-cleanup.ts` (parts-bot TTL deletes). Manual-invoke only until added to `vercel.json` .

6.2 GitHub Actions (Mechanics Desk workers — 16 workflows)

MD has no API; these Playwright workers log in with `MECHANICDESK_{WORKSHOP_ID,USERNAME,PASSWORD}` secrets and talk back to the portal with `X-Service-Token: $JA_PORTAL_API_KEY` . All install Chromium at run time (`setup-node@v5` + `npx playwright install chromium` — the old "run in the MCR container" note in `05_INTEGRATIONS.md` is obsolete). **`playwright` in devDependencies must match the installed version (1.59.1).** MD allows a single session per employee — concurrency groups prevent workers evicting each other.

Workflow	Schedule (Brisbane) / trigger	Job
`mechanicdesk-pull`	08/10/12/14/16:00	MD WIP report → forecast ingest
`md-stock-sync`	every 30 min, 06:00–19:30 Mon–Sat	Full MD stock catalogue → `md_stock_cache` (Slack parts bot)
`md-workshop-map`	03:30 nightly	Full invoices/quotes/customers pull → workshop map (FY2025 was a one-time backfill; nightly FROM = 2025-07-01)
`mechanicdesk-prepick`	06/11/15:00 weekdays	Diary-job parts demand → Pre Pick snapshot
`md-oncar`	05:40 Mon–Sat, + on demand	Parts on STARTED jobs → `md_oncar_*` (“On cars” panel on Stocktake (MD))

Workflow	Schedule (Brisbane) / trigger	Job
`sales-recap`	Mon 07:00 full+email; ~7 intraday MD refreshes	Weekly Sales Recap (Monday boards + MD scrape → email Ryan)
`md-customer-import`	02:00 nightly	Monday quote-channel leads → MD customers (MD can't search phones)
`tune-jobs-md`	02:30 nightly	Submitted tune jobs → MD customer + vehicle + note
`mechanicdesk-stocktake`	`repository_dispatch`	Stocktake match / push / recheck / refresh / push-missing
`md-purchase-order`	`repository_dispatch`	Raise MD PO after a JAWS→VPS stock transfer
7 × `probe-md-*` / `fix-md-customer-names`	manual	Endpoint recon probes and one-off repairs

6.3 The print centre (comms room)

PORTAL-CENTRE is a dedicated always-on Lenovo ThinkCentre in the comms room whose only job is printing. It exists because the print agent used to run on Chris's MSI laptop, which meant **every letter, invoice and label silently stopped whenever the laptop was closed or off the network** — with no error anywhere, because the queue simply stopped draining. Built and verified 2026-08-20.

Host	Lenovo ThinkCentre, comms room, always on (no lid, no sleep)
Reachable	Tailscale `100.72.189.95` — RDP in for maintenance; it is Entra-joined, so sign in with the work account
Runs	`agents/label-print-agent/` on Node 22, started by Autologon + Startup shortcut, headless
Queue	`label_print_jobs` in Supabase, consumed over Realtime with a 30s poll as fallback
Job kinds	`label` (DYMO 4XL), `invoice`, `letter`, `envelope` (Apeos) — all four verified end to end
Runbook	`docs/print-centre-thinkcentre-setup.md` — build, headless operation, the comms-room move sequence, RDP, and a health-check script

Operating notes

- The agent **never restarts on deploy** — it is not part of the Vercel app. A new print kind must not require an agent change; add it as a queue row shape the agent already understands.
- A stuck queue drains on agent restart, so restarting the agent is the first fix. Failed jobs are re-queued by flipping `failed` → `pending`.
- ⚠ **Never trust the Apeos `(Copy 1)` printer-name suffix.** Windows appends it when a printer is re-added, and printing to the wrong name fails silently or prints to the wrong device — check the real name.
- Printer enumeration must never take the heartbeat down (fixed 2026-08-20) — if the agent looks dead but the box is up, check the heartbeat before rebuilding anything.
- DYMO troubles: the health-check script has label-printer diagnostics and a `-FixDymoPort` switch.

6.4 On-premise agents

Agent	Where	Notes
`label-print-agent` (`agents/label-print-agent/`)	**PORTAL-CENTRE** — see §6.3	Consumes `label_print_jobs` via Supabase Realtime (30s poll fallback). Kinds: `label` (DYMO 4XL), `invoice`, `letter`, `envelope` (Epson). **Never restarts on deploy** — new print kinds must not require agent changes. Runs via Startup shortcut → `run-agent-hidden.vbs` or NSSM. Node 22, Autologon, reachable on Tailscale `100.72.189.95`; built 2026-08-20, runbook `docs/print-centre-thinkcentre-setup.md`. Previously ran on Chris's MSI laptop and failed silently whenever it was off-network — that is what the dedicated box fixes. **Never trust the Apeos `(Copy 1)` printer-name suffix.**
PBX workers	FreePBX box	See §5.10. systemd units; Node 16 for the camera bridges.
ESP32 scale nodes	Workshop bins / cash-count rig	Firmware `hardware/ja-scale-node/`; raw HX711 counts to `/api/scales/ingest`; all calibration server-side.

---

7. Modules & SOPs

Full route-by-route inventory: 117 page routes (444 API routes). Staff nav is the `/home` app launcher (role- and `visible_tabs`-filtered). Below, per module: what it is + how to operate it.

7.1 Dashboards (`/dashboard`, `/overview`, `/home`)

Section-based management dashboard (JAWS+VPS merged, entity filter pills) and a drag/resize custom dashboard builder with per-user saved layouts. **SOP**: nothing operational — data flows from MYOB reporting caches; if figures look stale, check the 02:00 `distributors/refresh-cache` cron and the health page.

7.2 Workshop tooling (Mechanics Desk stays the system of record)

Mechanics Desk is the workshop system. The diary, job cards, customers, vehicles, quotes and invoicing all happen in MD, and there is **no migration in progress** (decision 2026-08-20). The portal does not run the workshop.

What the portal *does* provide around MD, all of it in daily use:

Surface	What it does	Fed by
Letters (`/workshop/letters`)	Hourly `letter-watch` cron queues a thank-you letter + envelope for each newly finalised MYOB invoice and prints them on the Apeos. Deposit-only invoices are vetoed by a positive 1-1230 line. Reprint / compose / edit templates here.	MYOB (not MD)
Pre Pick (`/workshop/prepick`)	14-day parts demand vs stock on hand.	`mechanicdesk-prepick` Playwright worker
Purchase Orders (`/workshop/purchase-orders`)	PO draft → sent → received, with a bill push to MYOB.	Portal + MYOB
Stocktake (MD) (`/stocktake`)	Count sheet reconciled against MechanicDesk.	MD workers

Surface	What it does	Fed by
Stocktake (Portal) (`/workshop/stocktake`)	Barcode session → count → Variance → Apply.	Portal
Stock Transfer (`/admin/b2b/stock-transfer`)	JAWS↔VPS paired invoice + bill, plus the MD PO.	Portal + MYOB + MD

The parked sections are switched off in the UI (2026-08-20). `lib/workshop-sections.js` is the single source of truth for which sections are on; the tab strip reads it, and `next.config.js` reads the same list to rewrite the parked routes to a "not in use" notice.

- Off: Diary, Jobs, Customers, Vehicles, Quotes, Invoices, Comms, Workshop Reports (plus their `/workshop/{job,customer,vehicle,quote,invoice}/[id]` detail routes), and — second wave, same day — Orders, Inventory, Live Bins, Cash Count, Suppliers. 18 routes in total.
- On: the six in the table above.
- **The nav is now a single flat strip.** `components/InventoryTabs.tsx` (the old Inventory second level) was deleted once Inventory/Live Bins/Cash Count/Suppliers/Orders came off — there was almost nothing left to wrap — and its survivors were promoted to the top strip.
- The Workshop app tile opens `/workshop/prepick`.
- Parked routes **rewrite**, not redirect — the URL stays, so an old `/diary` bookmark still reads `/diary` and explains itself. The rewrites are in `beforeFiles`; an array return or `afterFiles` is evaluated *after* the filesystem and the real page would render instead.
- `?preview=1` on a parked route skips the rewrite and loads the real page — an escape hatch for anyone reviving the build.
- **To switch a section back on: flip `active` to true in `lib/workshop-sections.js`.** Nothing else. `docs/workshop_md_parity.md` still holds the parity and cutover detail.

⚠ Because MD stays, the **9 scheduled MechanicDesk scrapers remain a live production dependency** (§6.2), not a temporary bridge. Treat them accordingly — an MD password change or UI change breaks real reporting.

7.3 B2B — distributor portal (`/b2b/*`)

Distributor experience: Shop → Cart (PO number required, ≤20 chars — MYOB limit) → Stripe Checkout (card+surcharge / PayTo / BECS) → Orders (status timeline + freight tracking) → Jobs (tune receipts to fill in) → Resources → Training → Team → Settings.

Call coaching in Slack: 120 cards a day became one section of one post (2026-08-31). Two changes the same day. First, `#sales-coaching` was receiving a full coaching card per analysed call — **127 on 31 August**, 100-140 on any weekday. Chris: "send 1 message at the end of the day with a recap of coaching notes for that day". Per-call cards went **off by default** (`CALLS_COACHING_PER_CALL_CARDS=1` brings them back without a deploy) and a daily recap cron posted one message at 18:05. Then, hours later: *"One channel to get all your information"* — so that recap was **folded into the 5:15pm sales update** and its cron deleted. Nothing is lost from the coaching itself: every call is still transcribed, scored, attributed and readable at `/calls` — only the Slack delivery changed.

`lib/calls-daily-recap.ts` is now a **section provider, not a poster**: `buildCoachingSections(now)` returns `{ blocks, textLine, result } | null` and owns no channel. `postDailyRecap` and `pages/api/cron/calls-daily-recap.ts` are gone.

Four sections, all four requested: **top call** (with a portal link), **the themes that recurred**, **a per-advisor line** (calls, average, weakest dimension), and **one worth reviewing** (the weakest call).

- **Figures come from `fetchCoachingWindow()`** — the Monday report's own assembler, taking an optional explicit window. The weekly path calls it with no options, so its behaviour is byte-for-byte unchanged.
- **The themes are SUMMARISED, not counted.** This is the trap: the analyser writes a long, unique paragraph per call ("The close was almost invisible. After Ben confirmed he wanted a quote…"), so a frequency count over those strings returns every item at n=1 and would have printed five arbitrary essays. Checking the actual data before shipping is what caught it. The recurring patterns are real — soft closes, shallow discovery, incomplete details captured — but they have to be read out of the prose, so a bounded LLM call (60 items, 320 chars each, `max_tokens` 1200) extracts them. **Fails open**: no key or any error omits that one section and the rest of the post still goes.
- **A window with nothing scored adds nothing.** No "0 calls" line.

The coaching window is 16:30 → 16:30 Brisbane, and that is not an arbitrary choice. `fetchCoachingWindow` filters on `calls.call_date` — when the call **started** — and joins the analysis by call id, so a call whose analysis hasn't landed is silently *absent* rather than pending. Analysis lags the call by **22.5 min mean, 27 min p95**. On a calendar-day window a 16:55 call sat inside today's window but was unanalysed at 17:15, and tomorrow's window began at midnight — so it was coached in **no** recap at all, about 1-2 scored calls a day. Call *start* time was never the issue: over 21 weekdays, of 1353 scored calls exactly **one** started after 17:15. The 16:30 cutoff buys a 45-minute settle margin, and both edges are anchored on the calendar date rather than on `now`, so consecutive posts abut exactly — nothing double-counted when a pass lands late, nothing dropped in a gap. The lower edge steps back to the previous **weekday**, so Monday's post reaches Friday 16:30 and covers the weekend. Late calls appear in **tomorrow's** post.

An intermediate design — window from 17:15 yesterday to `now` — was proposed and rejected during the work because it does not actually close the hole: a 16:55 call is unanalysed inside `[17:15 yest, 17:15 today]` and outside `[17:15 today, …]`, so it stays lost. The bug is the `call_date` filter, not the window length; only a cutoff *earlier than the post* fixes it.

Daily 5:15pm sales update to Slack (2026-08-31). `lib/sales-update-slack.ts`, posted by `/api/cron/sales-update` to **#sales-updates**. Monday to Thursday it posts the day; **Friday it posts the week** with a Mon–Fri breakdown. Weekends are skipped — an empty post every Saturday trains people to ignore the channel. Since the same day it also carries the **call-coaching recap** (above), making it the one end-of-day post.

Every figure comes from `fetchSalesFigures()` in `lib/sales-figures-monday.ts`, the same source as Reports → Sales Report, so the Slack post and the report cannot disagree. "Sales" means **orders written**, not turnover invoiced — that is what the Monday boards hold. `ordersValue` is Just Autos bookings (Orders board), `distValue` is the Distributor Booking board, and `people` is already sorted by total descending, so the top seller is `people[0]` — ranked on **combined value, orders + distributor** (Chris's call).

- **Targets are per stream and per day**: `SALES_TARGET_JA_PER_DAY` ($60,000) and `SALES_TARGET_DIST_PER_DAY` ($50,000), combining to $110,000 (Chris, 2026-08-31). The post shows each stream against its own target and then the combined figure. Both keys are in `INTEGRATION_KEYS`, so they

really are editable at Settings → Integrations — note `SALES_TARGET_PER_DAY`, used by the weekly recap, is **not** in that list and is env-only, so the two targets are deliberately separate keys rather than a reuse. The Friday post multiplies each by the weekdays covered, so a full week is judged against a full week's target.

- **Channel** is `SALES_UPDATE_SLACK_CHANNEL`, defaulting to the id `C0BTL0TND6X` (#sales-updates) — an id rather than a name so it survives a rename, and a portal setting so it can move without a deploy.
- `renderSalesUpdate()` **is pure** — no network, no clock of its own — so the exact message can be checked against known figures without Monday or Slack credentials. Splitting it out immediately caught two bugs: the plain-text fallback was built by stripping `/[*_]/`, which also ate the underscores inside emoji shortcodes (`:bar_chart:` → `:barchart:`) in the line Slack shows in notifications; and the weekly heading used *today* as its end date rather than the period end, so a forced `?mode=weekly` preview mid-week read "Monday to Monday".
- **Scheduled hourly from 17:15 to 21:15 Brisbane, not once a day, deliberately.** A once-daily cron that collides with a deploy is skipped silently — the exact failure that cost the tune-job chase a whole week, diagnosed the same day. `app_settings.sales_update_last_posted` holds the Brisbane date, so the first pass from 17:15 posts and later passes do nothing. **The marker is only written once Slack accepts the post**, so a Slack outage retries on the next hour instead of losing the day.
- `/api/admin/sales-update-preview` (`admin:settings`) composes the message and returns it **without posting** on GET — including the raw figures and the `coaching` outcome, so the numbers can be checked against the report at any time of day. POST to the same route sends it immediately. `?mode=daily|weekly` forces either shape. Its `maxDuration` is 300 too, since it now runs the Anthropic call as well.
- **The coaching part fails open in two layers**: the whole coaching build is wrapped in `buildSalesUpdate`, and the Anthropic theme call has its own catch inside `calls-daily-recap`. A Supabase, Anthropic or coaching failure drops that section and **the sales figures still post**. The outcome is reported rather than swallowed — `SalesUpdate.coaching = { included, callsAnalysed, reason }` comes back from `postSalesUpdate`, in the cron's JSON and in the preview — so "nothing scored in the window" is distinguishable from "the Anthropic call died" without reading function logs.
- **Friday is deliberately asymmetric**: the sales figures are the WEEK, the coaching is still only its one 16:30→16:30 span. There is no weekly coaching mode — the Monday report covers the week — and the coaching heading names its exact span so the two cannot be confused.
- `renderSalesUpdate` **stays pure** after the merge: the coaching blocks are fetched in `buildSalesUpdate` and passed in as an argument, so the message can still be verified against known figures with no credentials.

The weekly tune-job chase is a WINDOW now, not one run (2026-08-31). It used to fire only where `bris.getUTCDay() === 5 && bris.getUTCHours() === 8` - a single hourly pass per week, at 22:20 UTC. Miss that one run and the entire week's distributor chasing AND the recap to Matt were skipped, silently, with nothing to notice it by. That is what happened on **Friday 28 August**: `last_reminder_at` stopped at 2026-08-20 22:21, 237 jobs went unchased and Matt got no recap, and nobody knew until he said so three days later.

The run itself vanished without a trace: `notified_at` stamps exist at 23:20:54Z and 04:20:39Z either side, but nothing at 22:20Z, so that pass had no effect at all rather than failing part-way. A production deployment was created at **22:18:59Z, 61 seconds before it** (one of seven that hour), and Vercel re-registers crons on deploy, so a changeover collision is the most likely cause. A transient throw is the other candidate and cannot be ruled out from here - runtime logs are long past retention. The fix removes sensitivity to both.

- **Window**: Friday 08:00 Brisbane through Sunday. Any hourly pass in that window will do the work.

- **Marker**: `app_settings.tune_jobs_weekly_chase_friday` holds *that Friday's Brisbane date*, so it happens once per week however many passes fall inside the window.
- **Written as `daysSinceFriday = (getUTCDay() - 5 + 7) % 7`, deliberately not a `getUTCDay() >= 5` test** - Sunday is `0`, so the `>=` form silently drops the last day of the catch-up window.
- **The marker is set only once BOTH the reminders and the recap have gone.** A failed recap therefore retries next hour, and the reminders that re-run alongside it send nothing new: `sendTuneJobReminders` only picks jobs whose `last_reminder_at` is null or older than 6.5 days, so re-entry cannot chase a distributor twice in a week.
- **Monday starts a new week.** A week missed entirely is not chased late - the catch-up is deliberate, not open-ended.
- `ingestTuneJobEmails` is now wrapped in `.catch()` like every other step. It was the one unguarded call in the handler, so a Graph hiccup would have taken the weekly block down with it.

The "Send reminders now" button sends the recap too (2026-08-31). It called `sendTuneJobReminders()` alone, so the manual remedy chased distributors and still left Matt with nothing - the one path most likely to be used *because* a scheduled run was missed. It now runs both and returns `recap_sent` so the operator can see it happened.

Related risk, not changed: `/api/cron/calls-weekly-report` (`0 21 * * 0`) and `/api/cron/workshop-map-weekly` (`10 21 * * 0`) are also once-a-week single slots, and a deploy landing on them would skip them the same way. They are scheduled by Vercel rather than gated in code, so they have no marker to retry from.

Tune-job recap to Matt (2026-08-26). `sendTuneJobRecap()` in `lib/b2b-tune-jobs.ts` runs from `/api/cron/tune-jobs` straight after `sendTuneJobReminders()`, i.e. on the Friday 08:20 Brisbane pass and on any manual `?remind=1` / "Send reminders now". One email to `TUNE_JOBS_RECAP_EMAIL` (default `matt.h@justautosmechanical.com.au`) with per-distributor outstanding count, oldest-job age, dollar value and whether they were chased. It reports EVERY distributor with jobs outstanding, not only those emailed - a distributor with no portal login receives no reminder and would otherwise be invisible, so they are called out explicitly. Uses the same `tuneJobVisible()` delay filter as the reminders so it can never quote a job the distributor has not been shown. Wrapped in `.catch()` in the cron: a recap that fails must not cost the reminders that already went. `tuneJobRecapRows()` is exported separately so the figures can be checked without sending.

Tune jobs: the two filters now cross-cut (2026-08-26). The status tabs counted `jobs` (everything) while the table rendered the distributor-filtered set, so a tab's number bore no relation to the rows under it. Both directions are filtered now: status counts come from the distributor-filtered set, distributor counts from the status-filtered set, and the selected distributor stays listed at zero rather than vanishing. Empty status tabs hide, with `all` and the current selection always kept so an empty pick cannot strand you. **`unmatched` and `submitted` were NOT removed despite being empty in all 561 rows** - both are reachable (`b2b-tune-jobs.ts:389` and `:492`) and `submitted` is a transient the MD sync worker drains in seconds, so a pile-up there is the signal that the worker has died. Deleting the tab would hide exactly the failure it exists to show.

Stock EOM: per-SKU averages start at the first selling month (2026-08-26). `avgUnitsPerMonth` / `avgRevenuePerMonth` divided by the FULL history window, so a SKU first sold last month was averaged across months it did not exist in - JA-VD300-1BB read 1.17/month instead of 7.00 on the July report, a 6x understatement on the figure the reorder list is built from. The denominator is now `series.length - firstSellingIndex`, with `historyMonthsUsed`, `historyFromMonth`, `historyPartial` and `firstSold` (earliest sale in the fetched range, month-end bounded like `lastSold`) exposed on `EomItem` so every surface

can say which averages rest on a short window. `growthPct` is forced null when the used window is under half the full one - `halfOverHalfGrowth` already returned null for an empty first half, this covers a first sale landing mid-first-half. **Stored snapshots keep the old numbers until rebuilt**, since screen, PDF and email all serve the snapshot. "Units" is relabelled "Sold units" in the top-movers and margin tables on all three surfaces.

Drop-ship confirm watch: acknowledged is not dispatched (2026-08-26). `classifyConfirmation` returned a single `confirmed` boolean, and its prompt made "accepts/acknowledges the order" true - so a supplier saying "all in stock, will get it shipped out today" ran `receiveDropShipPo`, billing the PO and raising the DISTRIBUTOR'S TAX INVOICE days before anything left (MPI / B2B-2026-000052). The verdict is now a three-way `stage`: `dispatched` requires evidence the goods have LEFT (their invoice, consignment/tracking, freight details, or past-tense shipped); `acknowledged` covers acceptance and any future or same-day promise; `neither` is everything else. Only `dispatched` triggers the receive. `acknowledged` still records the ETA, emails the distributor, posts to Slack and writes a `dropship_acknowledged` event - and crucially **leaves the order in `candidates`**, so the later dispatch email still matches it. The classifier is told to prefer acknowledged over dispatched when unsure, because the failure mode of guessing wrong is invoicing a customer early.

MYOB requires `FreightTaxCode` on an Item Bill even when `Freight` is 0 (2026-08-26). Omit it and the POST is rejected outright with "FreightTaxCode is required". `convertDropShipPoToBill` only set it inside `if (Freight > 0)` - and a drop-ship PO never carries freight, because the supplier ships direct and bills separately - so **every** drop-ship bill had been failing (MPI PO 00001382, B2B-2026-000052). It now always sends `Freight` plus a `FreightTaxCode`, GST when there is freight and FRE when there is not, mirroring what the sale-invoice path already did. `lib/ap-myob-bill.ts:43` had documented this rule all along; the B2B and workshop purchase paths never picked it up. **`lib/workshop-po.ts` had the identical gap and was fixed pre-emptively** - it would have failed the first time a workshop PO was pushed to MYOB. If you add another `Purchase/Bill/*` POST, send both fields.

Drop-ship receive: four faults that compounded (2026-08-26). B2B-2026-000052 surfaced all of them at once. (1) `receiveDropShipPo` ran the sale-order -> invoice conversion **regardless of whether the PO bill succeeded**; for a drop-ship line the conversion consumes exactly the stock the bill receives, so an unbilled PO guarantees `Inventory_InsufficientStockMultipleLocation` - an inventory error that names nothing about the cause. It is now gated on every PO having `myob_bill_uid`, and skips with the outstanding POs named. (2) Bill failures were pushed into `steps` but **never written to `b2b_order_events`**, while invoice failures were - so the page showed the symptom and hid the cause. Now emits `dropship_po_bill_failed`. (3) The Slack alert in `b2b-dropship-confirm-watch` reads `run.error`, which the function never set on the completion path, so a failed receive posted "hit a snag" and nothing else. The first failing step is now returned as `error`. (4) `.slice(0, 500)` on MYOB error bodies kept the boilerplate and cut the specifics - the real event was truncated one character before the item name. `myobDetail()` collapses MYOB's pretty-printed JSON whitespace (which was eating the budget) and, if still over, keeps **both** head and tail.

ONE number for the portal order and the MYOB document (migration `214`, 2026-08-31). Two independent sequences used to run, so nothing cross-referenced: the portal order number (`b2b_orders.order_number`, `B2B-2026-000057`) advanced on **every** order including abandoned ones, while the MYOB number (`b2b_next_myob_invoice_number()`, `JAWSB2B0065`) advanced only on a successful write **and was also consumed by stock transfers**. Live drift when this was written: `B2B-2026-000057` → `JAWSB2B0065`, `B2B-2026-000050` → `JAWSB2B0059`, `B2B-2026-000049` → `JAWSB2B0055` — 6 to 9 out, and not by a fixed amount, so staff had to look an order up to translate between the number the distributor quotes and the number MYOB and accounts quote.

Now `b2b_next_order_number()` returns `'JAWSB2B' || lpad(nextval('b2b_order_seq'), 4, '0')` → `JAWSB2B0100`, allocated at order creation and posted verbatim as the MYOB `Number`.

- **Why 4 digits, not 6.** Two caps stack. MYOB's `Number` field is **13 characters**; and `lib/b2b-dropship.ts` stamps our number on the *supplier* PO, where suppliers 2..n get a `-2` / `-3` suffix and the result is `.slice(0, 13)`. A 13-char base (`JAWSB2B000100`) would have had the suffix **sliced clean off**, handing supplier 2 a PO number identical to supplier 1's — MYOB rejects the duplicate and falls back to its own meaningless sequential numbering, which is precisely what that feature exists to remove. `JAWSB2B` + 4 digits = 11, leaving exactly 2 for `-2` .. `-9`. (≥10 suppliers on one order would truncate again; MYOB's rejection → own-numbering fallback still catches it, and it has never happened — 0 multi-supplier drop-ship orders to date.)
- **Why the sequence jumps to 100.** `setval('b2b_order_seq', 99, true)`. MYOB already held `JAWSB2B0048` – `0065`, so continuing from the portal sequence's live value (61) would have made the unified series appear to run **backwards** over numbers already filed. The 0066–0099 gap is deliberate and marks the changeover.
- **The function raises above 9999.** `lpad` **truncates** rather than overflowing — `lpad('10000',4,'0')` is `'1000'` — so without the guard, order 10000 would silently mint a duplicate of order 1000 and kill checkout on the `order_number` unique constraint with no clue why. It is plpgsql purely to be able to refuse; widening the format must be deliberate, because 5 digits leaves only 1 character for the drop-ship suffix.
- **The `/^JAWSB2B\d{4}$/` guard** (`resolveMyobNumber`, used by `writeOrderToMyob` and the missing-number branch of `convertOrderToInvoiceInMyob`) posts `order_number` as the MYOB `Number` only when it matches; anything else falls back to `b2b_next_myob_invoice_number()`. Pre-214 orders are `B2B-2026-000053` — **15 characters, which MYOB rejects outright** — and two were live in `pending_payment` when this shipped. The guard also makes deploy-vs-migration ordering irrelevant. Verified against live data: **0 existing `order_number` values match `JAWSB2B%`**, so it cannot misfire on history.
- **Stock transfers moved to their own stream**, because they had to: the settings sequence sat at 65 and would have reached 0100 after only 35 more transfers, with 24 already done. New `b2b_settings.myob_transfer_number_{prefix,padding,seq}` (default `JAWSTFR`/4/0) and `b2b_next_myob_transfer_number()` — row-locked increment-and-read-back like the invoice/credit-note allocators, plus `greatest(padding, length(seq))` so a 5-digit sequence grows (`JAWSTFR10000`, 12 chars) instead of truncating into a duplicate. `JAWSTFR` shares no prefix with `JAWSB2B`, so collision is impossible by construction. **Transfers 25+ read `JAWSTFR0001` …**; the 24 already filed keep their `JAWSB2B00NN` numbers.
- **Management Dashboard exclusion widened.** The dashboard keeps B2B intercompany journals out of JAWS revenue/GP by matching the GL "ID No." against `exclusions.invoiceNumberPattern`, seeded as the literal `"B2B"` (migration 184, 6 chart rows). It is compiled as `new RegExp(pattern, 'i')`, so 214 widens the live rows to `"B2B|JAWSTFR"` — a `JAWSTFR####` transfer would otherwise rest solely on the `memoPattern` (`Stock transfer.*JA Portal`) arm, which depends on MYOB propagating our JournalMemo onto every GL line and has never been verified. Migration 184's own seed still says `"B2B"`, so a rebuilt environment relies on 214 running after it.

Stream	Allocator	Shape
B2B order **and** its MYOB sale order/invoice	`b2b_next_order_number()` (`b2b_order_seq`)	`JAWSB2B0100`+
Pre-214 orders / any non-matching order number	`b2b_next_myob_invoice_number()` (`b2b_settings`)	`JAWSB2B00NN` — fallback only
Stock transfers (25+)	`b2b_next_myob_transfer_number()`	`JAWSTFR0001`+

Stream	Allocator	Shape
Refund credit notes	`b2b_next_myob_credit_note_number()`	`CR000001` + — unchanged

New failure mode worth knowing. The number is now fixed at order creation instead of freshly minted per attempt, so a retried MYOB write reuses it. If a POST created the document but the portal failed before recording its UID, the retry gets a **duplicate-number rejection** and the order shows a `myob_write_error`. That is strictly better than the old behaviour (a second document under a new number) but it needs a human to link it. Adoption-by-Number — the pattern `lib/b2b-myob-po.ts` already uses for hand-created bills — would close it.

Drop-ship POs carry our MYOB invoice number (2026-08-26). `createDropShipPurchaseOrder` accepts an optional `number` and sets MYOB's `Number` field; `raiseDropShipPOsForOrder` passes `order.myob_invoice_number`. The pipeline writes the MYOB order (and so that number) before raising drop-ship POs, so it is always available. MYOB caps `Number` at **13 chars** and enforces uniqueness across purchases, so multi-supplier orders get `-2`, `-3` suffixes — and if MYOB still refuses the number, the create is **retried once without it** rather than losing the PO, with a warning naming the refused value. Before this, MYOB assigned a sequential number (00001382) that appeared on the supplier email and matched nothing at either end.

Drop-ship PO re-send used a different envelope to the raise (2026-08-26). The raise path sent `from: PO_FROM_MAILBOX`, `replyTo` + `cc: PO_COPY_MAILBOX`; `resendDropShipPoEmail` sent `sendMail(await getFromMailbox(), { to, subject, html })` - no CC, no Reply-To, different From. Resend deliveries never appear in a Sent folder, so that CC is the only evidence a PO left the building, and a re-sent one was invisible from our end while replies went to the wrong inbox. Both paths now share one envelope. `getFromMailbox` is no longer imported in that module.

Supplier emails: MYOB's Email field is free text (2026-08-25). `getSupplierContact()` returned the raw MYOB `Addresses[].Email` value trimmed, and it went straight into Resend's `to`. MYOB cards routinely hold several addresses in one box, and Resend 422s the entire send on anything that is not one clean address - which killed MPI AUTOMOTIVE's PO 00001382 email on 2026-08-25 after the same supplier had received one fine on 6 August. `lib/email-recipients.ts` `parseEmailList()` now splits on `;` / `,`, unwraps `Name <addr>`, strips trailing `(notes)` and `- notes`, recovers an unbracketed trailing address, de-duplicates case-insensitively, and returns bare addresses; drop-ship PO sends mail ALL of them. A value that yields nothing usable is reported as `failed` with the offending value quoted, NOT as `no_email` - those are different problems and were indistinguishable. Unit-checked against 13 real-world shapes. **Other callers of the same lookup were left alone** (`post-b2b-doc.getSupplierContact`, and the Xero branch) - if another emailer starts 422ing, it wants the same treatment.

"Check if payment cleared" (2026-08-25). `POST /api/b2b/admin/orders/{id}/check-payment` retrieves the Stripe PaymentIntent (falling back to the checkout session, and backfilling `stripe_payment_intent_id` when it was never stored) and treats ONLY `succeeded` as cleared - `processing` is a BECS debit still in flight, which is the state the button exists to distinguish. On cleared it reproduces the `async_payment_succeeded` webhook's three effects in the same order: stamp `payment_settled_at` guarded on null, write a `payment_settled` event, and `applyCustomerPaymentInMyob()` if the sale invoice exists. Guarding on null makes it idempotent and race-safe against the webhook - whichever lands first wins and the other reports "already recorded". A MYOB failure does not fail the check: settlement stands and the note says so. Exists because settlement had exactly ONE source (the webhook), so a dropped delivery left an order unsettled forever, blocking both the Ship Now credit gate and the MYOB receipt with no way to ask.

Carriers are now filtered by what they can physically carry (2026-08-31). There was **no carrier filtering anywhere** — `quoteLiveRates()` offered every route MachShip returned, and `pages/b2b/cart.tsx` auto-selects the cheapest, so a consignment of loose cartons could be pre-selected onto a pallet-only linehaul carrier without anyone choosing it. Chris: "Hi trans wont send individual consignments so that needs to be set as a rule."

`b2b_freight_carrier_rules` (migration **213**) is checked in `quoteLiveRates()` **before** the cheapest-per-carrier collapse — an ineligible carrier never reaches the rate list, because merely appearing there is enough for the cart to auto-select it.

- `pallets_only` — offer this carrier only when every item in the candidate packing is a `Pallet` / `Skid`. Seeded true for Hi-Trans.
- `blocked` — never offer at all. A kill switch that needs no deploy.
- Matching is **case-insensitive substring on the carrier name**, because MachShip's carrier ids are not documented anywhere we control and a rename is likelier than an id change. `machship_carrier_id` can pin an exact id once one is observed, and wins over the name when set.
- **Three spellings are seeded** — `hi-trans`, `hi trans`, `hitrans`. The separator-less form matched nothing in testing, which would have silently disabled the rule; MachShip is inconsistent about this.
- **Fails OPEN**: if the rules table cannot be read, quoting proceeds unfiltered and logs it. Losing every carrier would stop checkout dead, which is worse than briefly offering one we would rather not.
- If every route is excluded the quote says so explicitly, rather than reporting "no routes available" and sending someone hunting a MachShip or address fault.

Nothing had shipped wrongly. All 14 booked orders to 28 August were carrier 11 / service 540, TNT Road Express — the Hi-Trans exposure was in quotes only.

NOT DONE — split shipments across carriers. Chris also wants the quoter to compare "pallet with one carrier, loose items with another" against "all with one carrier". That is not a filter, it is a second consignment: **68 references across 16 files** assume one consignment per order (`machship_consignment_id` and friends on `b2b_orders`, with no consignments table). It needs a consignments table, a booking loop, per-carrier manifesting, per-consignment tracking and despatch state, distributor emails carrying several tracking numbers, and labels per consignment. The pick list already sections by consignment, so that part is ready. Deliberately left as its own piece of work rather than half-built.

Basemap moved off CARTO (2026-08-31). Both Leaflet dashboards — Reports → Distributor Map and Workshop → Map/Conversion — drew tiles from `basemaps.cartocdn.com/dark_all`. CARTO began requiring an API key and now **watermarks unauthenticated tiles with "API KEY REQUIRED" painted into the tile images themselves**, so the message appeared diagonally across both maps. Nothing of ours was misconfigured and no data was affected — the tiles still served 200s.

Now on Esri's key-free **Dark Gray Canvas**, via one shared helper, `lib/map-basemap.ts`. It is shared deliberately: the two dashboards each carried their own copy of a single tile URL, which is why one provider change broke both and would have had to be fixed twice.

Three things in that file are load-bearing:

- **Esri's path order is** `{z}/{y}/{x}`, not Leaflet's usual `{z}/{x}/{y}`. Writing it the conventional way serves the wrong part of the world without erroring.

- **Detail stops at zoom 16.** Past that Esri returns a light-grey tile reading "Map data not yet available" — which would simply be a *different* message written across the map. `maxNativeZoom: 16` makes Leaflet upscale zoom 16 instead, so deep zoom stays dark.
- **Esri splits base imagery from place names** (CARTO's `dark_all` combined them), so labels are a second tile layer. The distributor map takes both; the **workshop map takes the base only** (`labels: false`) because it draws its own curated state/city labels in `lblPane` and covers the tiles with filled land polygons — a second set of names would fight with them.

Esri's terms require attribution, so both maps now enable Leaflet's attribution control (`setPrefix(false)` drops Leaflet's own plug). That small credit bottom-right is new; it is a licensing requirement, not decoration. If the look ever needs to go back to exactly CARTO, a free CARTO key appended to the old URL restores it — the alternative that was weighed and declined.

Reminders — carts, unfinished checkouts and orders we're sitting on (2026-08-31). `lib/b2b-reminders.ts`, driven by `/api/cron/b2b-reminders` every 3 hours. Three independent passes; one throwing never stops the others, and every pass is idempotent so an extra run sends nothing twice.

- **Abandoned cart → distributor.** 24 h after the cart was last touched, and again at 72 h, then silence. A cart's last-touched time is derived from `b2b_cart_items` (`added_at` / `updated_at`), *not* `b2b_carts.updated_at` — the cart item routes bump the item rows and leave the cart's own stamp alone, so trusting the cart would call an actively-used cart abandoned. Comparing the reminder stamp against that same item timestamp is also what re-arms the cycle: touch the cart and the stamp is older than the newest item, so it legitimately becomes a candidate again — no reset logic needed. Guard columns `reminder_24_at` / `reminder_72_at` (migration **211**). Carts untouched for more than **14 days** are left alone: that is furniture, not an opportunity, and a "freight was quoted over 24 hours ago" warning reads absurdly against a month-old cart. Goes to the person whose cart it is (if their login is still active), else the distributor's primary contact.
- **Checkout started, never paid → distributor.** Once, 24 h in. Guarded hard, because chasing someone for money they already paid is the worst outcome available: Stripe is re-asked directly and a session that actually completed is flagged rather than chased (the SOP's own rule); a **later paid order from the same distributor** means they simply went round again (the 000051→000052 pattern) and is skipped; test orders never chased. Both existing candidates at build time were Banana Coast checkouts abandoned four minutes apart and paid later — the guard correctly skipped both.
- **Paid but not shipped → us.** At 2 days, escalating once at 5. Slack via `postB2bOrderSlack` plus a portal bell to admin/manager. The message says *why* it looks stalled — no freight booked / booked not shipped / manifested not marked shipped / waiting on a named drop-ship supplier — because an order waiting on a supplier is a different problem from one nobody has picked.

The once-only guards for the two order passes are `b2b_order_events` rows (`checkout_reminder_sent`, `stall_reminder` with a `stage` in metadata), so only the cart pass needed a migration. Wording for all three lives in `TEMPLATE_DEFS` (`distributor_cart_reminder`, `distributor_cart_reminder_final`, `distributor_checkout_unfinished`) and is editable at Admin → B2B → Email templates like every other B2B email; the *cadence* is constants at the top of `lib/b2b-reminders.ts`.

On rollout every existing cart was stamped as already-reminded, so the feature acts only on carts abandoned from that point rather than blasting a backlog — five carts (two of them a fortnight old) would otherwise have gone out at once. Clearing the two columns on a cart re-arms it.

All payment surcharges stop on a DATE, not on someone remembering (2026-08-31). Chris: the card surcharge comes off 1 October; on the question, **all** methods rather than card alone. `b2b_settings.payment_surcharge_ends_on` (migration **212**, set to `2026-10-01`) is checked by `surchargesEnded()` in `lib/b2b-payment.ts`, and on/after that Brisbane date card, PayTo and BECS all return zero.

- **Compared on the Brisbane calendar date, not UTC.** 1 October in Brisbane starts at 14:00 UTC on 30 September; a UTC comparison would still have charged a distributor checking out at 09:00 on the 1st. Boundary-tested at 30 Sep 13:00/23:59 Brisbane (charged) and 1 Oct 00:00/09:00 (not charged).
- **The rates themselves are left alone.** `card_fee_percent` / `card_fee_fixed` keep their values, so the decision is reversed by clearing the date rather than by someone recalling what 1.7% + $0.30 used to be. Editable at Admin → B2B → Settings → Card Surcharge, which now also shows whether the date has passed.
- **Gated in all three places a surcharge is computed** — `checkout/start.ts`, the cart estimate, and `admin/test-order.ts`. Missing any one of them would make a test order or a cart disagree with what is charged.

The cart estimate was hardcoded (found and fixed doing the above). `pages/api/b2b/cart.ts` carried its own `CARD_FEE_PCT = 0.017` / `CARD_FEE_FIXED = 0.30` and never read `b2b_settings`, so zeroing the surcharge in Settings would have left the cart quoting 1.7% + 30c while checkout charged nothing — a discrepancy that would have surfaced on 1 October in front of distributors. It now reads the same row as checkout, falling back to the old constants only if the row cannot be read, so a settings failure can never quote zero and then charge.

A settled payment now reaches MYOB no matter when it settles (2026-08-31). BECS order `B2B-2026-000050` / `JAWSB2B0059` cleared on 27 Aug and sat for four days with $4,074.46 never receipted into MYOB. Three separate recovery paths all missed it, because each was keyed on `myob_sale_invoice_uid` — a field that is only populated once the order **ships**:

- the settlement webhook applied the payment only `if (o?.myob_sale_invoice_uid)`. 000050 settled at 00:20 and shipped at 03:33, so the gate was shut when the money landed and nothing re-checked afterwards;
- the **Check payment** button returned early ("already recorded as cleared"), so there was no manual trigger either;
- the `b2b-payment-check` cron filtered on `myob_sale_invoice_uid is not null`, so it could not see the order at all.

The gates were all stricter than the function they guarded: `applyCustomerPaymentInMyob()` already falls back to the open Sale Order (MYOB books it as a customer deposit and carries it onto the invoice at conversion) and has its own correct preconditions. All three now call it unconditionally and let it decide.

`applyCustomerPaymentInMyob()` also stopped trusting the stored UID. At checkout we record the Sale **Order** UID in `myob_invoice_uid`; converting it consumes that document. Our converter mints a new invoice UID and writes it back, but a conversion done **by hand in the MYOB UI writes nothing back**, leaving the order pointing at a document that no longer exists. `resolvePaymentTarget()` in `lib/b2b-myob-invoice.ts` resolves the live document instead, in order: the known invoice UID → the stored UID as an Order → the same UID as an Invoice (a UI conversion that kept it) → the invoice whose `Number` matches ours (one that didn't). A newly discovered invoice is written back to `myob_sale_invoice_uid`, so the other callers stop chasing the dead UID. A `Number` that matches two invoices is treated as no match — guessing would post to the wrong one.

The cron gained a second pass keyed on **the money, not the shipping state**: settled, no `myob_payment_uid` / `myob_payment_at`, not cancelled/refunded/test, and settled more than 15 minutes ago so it never races the webhook. It notifies admins when it applies one, because reaching that pass means the primary path missed. The application is bounded by the document's live balance (`Math.min(balance, total_inc)`), so it can never overpay, and is idempotent on `myob_payment_uid`.

The button on the order page stays visible when an order is settled but unreceipted, reading **"Receipt payment in MYOB"** — previously the repair had no trigger at all. It is hidden on cancelled/refunded orders (a refund mirrors as a separate credit note, so the original invoice can still show an open balance) and once `myob_payment_at` is stamped, which covers `invoice_already_paid` — a hand receipt in MYOB sets the timestamp but never a `myob_payment_uid`.

A converted Sale Order still answers 200 (2026-08-31). The first live repair attempt on 000050 was rejected by MYOB with `CustomerMismatch` / ErrorCode 11007 — "Customer on payment must match customer on Order/Invoice" — despite the customer UID we posted (`2fc16db7…`) being **identical** to the one on the document MYOB returned. The message is misleading (MYOB's own response admits "error messages have not been finalised"). The resolver had stopped at the stored UID because `Sale/Order/Item/{uid}` returned 200 — but a converted order stays readable there and still reports a balance, while MYOB refuses payments against it. An order is now only a payable target when its `Status` is `Open` (or absent); anything else falls through to the invoice that replaced it. Payments against genuinely open orders — the normal card/PayTo checkout path — are unaffected, and had been succeeding all along (000044, 000049, 000056, 000057).

Two hardening changes came with it: the payment's `Customer.UID` is now taken from the **resolved document** rather than the distributor row, since that is what MYOB validates against (a divergence is logged as a data-integrity warning rather than silently papered over); and the `Number` branch of the resolver — the only one keyed on something we didn't record ourselves — requires the invoice to belong to the distributor being paid for, so it can never receipt a stranger's invoice. Rejections now name the document (type, Number, Status, customer, balance); diagnosing this one needed four SQL queries against `myob_api_log` because the error named nothing.

"Mark as shipped" is deliberately MYOB-free (confirmed by Chris, 2026-08-31). It records despatch and nothing else — no Order→Invoice conversion, no manifest. All MYOB work belongs to **Ship Now**, or to a conversion done by hand. 000050 went out via Mark as shipped, which is why it had no tax invoice and needed converting manually; that is the button working as intended, not a fault. What *was* a fault is that the money followed the invoice: since this change the payment reaches MYOB either way — against the invoice when one exists, otherwise against the open sale order as a deposit.

Orders list splits Status into Payment and Shipping (2026-08-25). The single `orderStatusLabel` pill conflated two independent axes - money in, goods out - so a green "Paid" on a boxed-but-unshipped order read as despatched. `paymentState()` / `shippingState()` in `pages/admin/b2b/orders/index.tsx` derive each separately. Payment goes amber, not green, while `payment_method` is `becs` / `payto` and `payment_settled_at` is null - those mark an order paid at mandate acceptance, days before the funds land, and Ship Now's credit gate keys off the same fact. Shipping reads the carrier's state via `awaitingDespatch()` from lib/b2b-despatch-state rather than our manifest id, and its vocabulary is deliberate: Book Freight creates a **pending consignment** (unmanifested - no carrier told, no collection booked), and **Ship Now** manifests it, which is what books the collection. Labelling the first state "Booked" told the warehouse a truck was coming for a parcel nobody had been asked to collect (Chris 2026-08-25). The states are No consignment / Pending consignment / Booked for collection / Shipped / Delivered / Consignment missing. The list API select gained `delivered_at`, `payment_method` and `payment_settled_at` to support it.

Abandoned checkouts are hidden from the admin orders list (2026-08-25). `checkout/start` must create the `b2b_orders` row before the Stripe redirect (the row id is the session reference), so backing out at the payment screen leaves a `pending_payment` row that looked exactly like a real unpaid order - B2B-2026-000051 sat at $8,081.26 while the same cart completed 27 minutes later as 000052. `pages/api/b2b/admin/orders/index.ts` now applies `.neq('status','pending_payment')` when no explicit status filter is given, to the list query, the value aggregate AND the `_all` tile count so the three agree. Filtering to the status still lists them - nothing is deleted. The label is now "Checkout not finished" in all three places it appeared. BECS/PayTo orders are unaffected: they are `paid` with a null `payment_settled_at`, a different thing entirely.

Still open: nothing ever *expires* these rows. Auto-cancelling on a new checkout was considered and rejected - a distributor with two tabs could pay via the older session, and the webhook would then mark a cancelled order paid. A cron that voids `pending_payment` orders whose Stripe session has genuinely expired is the safe version, and is not built.

Booking no longer marks an order shipped (2026-08-25). The 2026-08-20 split made Book Freight a preparation step — the consignment is created Unmanifested, nothing reaches the carrier, no tax invoice — but `lib/b2b-freight-book.ts` still stamped `status:'shipped'`, `shipped_at` and `shipped_by` on first book, thirty lines above its own comment saying it doesn't despatch. Every booked order therefore reported itself shipped, on the orders list and in the distributor's portal, while still on the bench. Booking now records `carrier` only; `shipNowForOrders()` owns the shipped stamp. The `freight_booked` event no longer claims `to_status:'shipped'` either.

⚠ `GET /apiv2/consignments/{id}` **IS NOT A REAL MACHSHIP ROUTE.** It 404s for every consignment that has ever existed. The freight poller hides this: it catches the 404 and recovers through `returnConsignmentsByCarrierConsignmentId` / `returnConsignmentsByReference1`, logging `re-resolved 71024867 -> 71024867` — the SAME id, which is the proof the id was never stale and the route is simply wrong. The "dead consignment" comments in `b2b-machship-refresh` are a misdiagnosis of that. **Do not build anything new on that GET.** Ship Now used to, which is how manifesting broke on 2026-08-27: it took CompanyId from that response, so CompanyId was always null and manifests failed the moment MachShip started enforcing it (orders up to 000050 went through; 000052/55/56 could not).

CompanyId resolution (`lib/b2b-ship-now.ts`), cheapest first: `b2b_orders.machship_company_id` (captured at booking, migration 207) → the `companyId` inside the stored `freight_chosen_quote` (every order has carried it since 2026-08-06 — nothing was reading it) → `b2b_settings.machship_company_id` (account-wide fallback, currently **50093**) → a live lookup by tracking number / Reference 1, written back to the order. All four missing fails the order with an actionable message rather than MachShip's bare "CompanyId is required".

Distributor PWA install doc (2026-08-28). `docs/distributor-app-install-sop.md` + PDF, registered in `lib/library-docs.ts` as `distributor-app-install` — a distributor-facing sheet staff can forward unchanged. Worth knowing the mechanism it documents: `pages/_document.tsx` serves **two manifests**, switching on `url.startsWith('/b2b')` — `/manifest.json` (JA Portal, `start_url: /home`) for staff and `/manifest-b2b.json` (**Just Autos Wholesale**, `start_url: /b2b/catalogue`, `scope: /b2b`) for distributors. **A distributor who installs from the bare domain therefore installs the STAFF app** and lands on a page they cannot access — the single most likely support call, so the doc leads with it. Distributor push is live and separate (`/b2b/settings` → `enableNotifications` → `/api/b2b/notifications/push-subscribe`); on iOS it requires the app be added to the Home Screen first and iOS 16.4+. `docs/**` is already force-included in `outputFileTracingIncludes`, so a new Library doc needs no config change.

Freight quote screen (`/admin/b2b/freight-quote` , 2026-08-27). Add products, type a suburb and postcode, get live MachShip rates — a dedicated screen rather than a panel inside the test-order builder, which meant pricing a hypothetical job on a page whose purpose is creating orders. Reuses `/api/b2b/admin/freight-quote` unchanged except that it now returns `pack_label` / `pack_key` / `pack_units` per rate, so each price can be expanded into the consignments and the boxes on each pallet. Nothing is created or saved. `view:b2b` , new Freight quote tab (`truck` icon) in `B2BAdminTabs` . The distributor list endpoint gained `ship_suburb` / `ship_postcode` so choosing a distributor prefills an editable destination.

Editing the boxes an order ships in (2026-08-25). `pack-plan.ts` gained `action:'setbox'` alongside `combine` / `reset` : `{ index, box }` re-assigns ONE consignment to a configured box (or `''` for own packaging), carrying weight and contents across and warning when the weight exceeds the box's limit. The cartonizer picks the smallest box an item fits, which is frequently not what the warehouse uses — and the box dimensions are what MachShip prices and the carrier bills. Grouped pallet units (`quantity > 1`) are refused; pack mode governs those. The order page renders it as a **ships in** dropdown per consignment, and the panel is now "Boxes and consignments". That panel was gated on `hasLiveQuote && !hasConsignment` , so it vanished the moment freight was booked and never appeared on static/satchel orders - you could not even see what an order shipped in. It now always renders, and goes **read-only** once `machship_consignment_id` exists. The API enforces the same rule with a 409 rather than trusting the UI: a saved plan that no longer matches the lodged consignment would print the warehouse a pick list for boxes the carrier is not expecting.

Stock Wall and Suppliers retired (2026-08-26). `lib/b2b-sections.js` mirrors `lib/workshop-sections.js` exactly - plain CommonJS so `next.config.js` (which cannot import TS) and `B2BAdminTabs` read one list. `active: false` drops the tab AND adds a `beforeFiles` rewrite to `/b2b-not-in-use?section=<id>` , so an old bookmark keeps its URL and explains itself; `?preview=1` skips the rewrite for a peek. Flipping `active` back to true is the entire revival. **Nothing deleted** - pages, APIs and tables all remain; `b2b_suppliers` and `b2b_supplier_users` were both EMPTY, so this retired unused surface rather than a working tool. The Suppliers section also parks the supplier-facing `/b2b/supplier` route: leaving a login open for a module nobody administers is worse than closing it. The notice page lives at the root, not under /admin, so one page can cover both staff and supplier routes.

Multiple delivery sites per distributor (migration 204, 2026-08-26). `b2b_distributor_addresses` - one distributor entity, many ship-to points, for the several-stores-one-ABN case. Deliberately NOT a second distributor account, which would split their pricing, credit and order history. A partial unique index enforces exactly one `is_default` per distributor, and a check constraint refuses an active address with no postcode (freight is priced on it). Every distributor's existing `ship_*` was backfilled as their default, and those columns REMAIN the fallback.

The key reason this was cheap: freight booking, MYOB `ShipToAddress` , the invoice PDF and drop-ship POs **already** preferred `b2b_orders.shipping_address_snapshot` over the distributor record - nothing had ever populated it. `checkout/start` now snapshots the chosen site there (plus `ship_address_id` as the link back), so all four print the right place with no changes of their own. The snapshot stays the authority: an address edited or deactivated later must not rewrite a shipped order.

`b2b_carts.ship_address_id` holds the in-progress choice so the on-screen freight quote is for the destination they picked; `POST /api/b2b/cart/ship-address` sets it and **validates the address belongs to the caller's own distributor** - the id comes from the browser. Staff manage sites at `/api/b2b/admin/distributors/{id}/addresses` (GET/POST/PATCH/DELETE, `edit:b2b_distributors`); DELETE deactivates rather than removes, and refuses to orphan the default. Self-serve was rejected: where a distributor's goods may be sent is a credit decision.

Minimum order quantity (migration 203, 2026-08-25). `b2b_catalogue.min_order_qty`, the mirror of `max_order_qty` from migration 125. NULL = no minimum (reads as 1) and is stored as NULL rather than defaulted to 1, so "not set" and "deliberately 1" stay distinguishable. Two DB check constraints back it: `min_order_qty >= 1`, and `min_order_qty <= max_order_qty` — a minimum above the maximum makes an item unorderable, and the admin PATCH maps both violations to a plain-English 400 rather than leaking the constraint name. Enforced in `pages/api/b2b/cart/items.ts` (qty 0 exempt — that's "remove", and a minimum must never trap a line in a cart) and again in `checkout/start.ts`, because a cart line can predate the minimum being set. The catalogue tile starts its Add button at the minimum and the cart stepper floors there; where the minimum exceeds the effective cap the tile says so instead of offering an Add that always fails. Carried in the catalogue CSV as "Min Order QTY".

Quantity writes are debounced (2026-08-25). The cart used to POST *and* `await load()` on every stepper press with the row disabled — five presses meant five sequential full cart reloads (pricing, stock, volume breaks, freight), which is what "the + lags" was. Both the cart and the catalogue now show the new qty immediately and commit only the last value in a burst (`COMMIT_DELAY_MS` 450 / `QTY_COMMIT_MS` 400). The post-write `load()` stays: volume breaks, caps and freight are server-computed, so the line total cannot be derived client-side without inventing prices — which is why the total *dims* while a change is pending rather than being multiplied out locally. The catalogue also no longer adopts `j.line.qty` from the POST response (id only): that patch-back let a slow reply overwrite a newer press.

`Stepper` in `components/b2b/ui.tsx` now holds a **draft** while focused and commits on Enter/blur, not per keystroke. Committing per keystroke meant typing "10" briefly set qty 1, and — worse — clearing the box to retype parsed `0` and removed the line.

⚠ **GST rounding is inherent, not a bug.** Prices are stored ex-GST and most were derived from a round inc-GST figure, so they don't round-trip: the 20 L oil is $163.64 ex, and ×1.1 = $180.004. The client shows "each" rounded per unit (`incGst()`), while `pages/api/b2b/cart.ts` computes `lineSubEx = unitPriceEx * qty` and `lineGst = lineSubEx * 0.10` unrounded, so qty × each ≠ line total from qty 2 up (1–2c). **42 of 50 visible priced items** have ex-GST cents not ending in 0 and so drift. Do not "fix" this by rounding the line total to qty × rounded-each — the ex-GST subtotal is what reaches MYOB, and forcing the inc-GST figure would reintroduce the cent drift the go-live review removed. The real fix is pricing data: an ex-GST price whose cents end in 0. Design language: the Alloy kit (`components/b2b/ui.tsx`) — distributor portal refreshed 2026-08-12, staff admin brought onto the same kit 2026-08-20.

Pricing is GST-INCLUSIVE on every distributor-facing surface — catalogue tiles, cart lines, cart totals, freight options, order list, order detail and the order emails. Prices are *stored* ex-GST (`unit_price_ex_gst`, `trade_price_ex_gst`, `price_ex_gst`) and converted for display by a local `incGst(ex, taxable)` helper in `catalogue.tsx` and `cart.tsx` — taxable items +10%, FRE items as-is, which is why the flag matters and a blanket ×1.1 would be wrong. Audited end to end 2026-08-25; the one surface that still led with an ex-GST figure (the order-detail totals block, "Subtotal (ex GST)") now reads *Items (inc GST)* / *Freight (inc GST)* / *Total paid* with an "Includes $X GST" line, matching the cart. **Freight is folded into `subtotal_ex_gst` at checkout**, so the order detail recovers it as the remainder — `items inc + freight inc == subtotal_ex_gst + gst` — rather than re-taxing `freight_cost_ex_gst` at a fixed 10%, which would be wrong for any non-taxable line. Staff-facing admin surfaces (freight zones, dropship calibration, catalogue audit/export) deliberately stay ex-GST: that is how the costing is done, and they are labelled. The **tax invoice PDF** also stays ex + GST + total, as an ATO tax invoice must.

What happens automatically on payment (`lib/b2b-order-pipeline.ts`): order → paid; cart cleared; MYOB invoice written cent-exact inc-GST; consignment-first pick list printed; for drop-ship items a supplier PO is created and emailed; Slack notification; freight bookable via MachShip (admin button or login-less email link). **The tax invoice is NOT raised at booking** — see Freight & despatch below.

SOPs

- **Onboard a distributor**: Admin → B2B → Distributors → Add (live MYOB customer typeahead links the card) → open the distributor → Invite users (magic-link / welcome token email). Owners manage their own team after that.
- **Over-limit orders**: quantities above `over_limit_qty` route to a quote-or-dropship flow instead of straight checkout.
- **Pallets are a table** (`b2b_freight_pallets` , migration 206), mirroring `b2b_freight_boxes` — CRUD at `/api/b2b/admin/freight-pallets` , edited in Settings → Freight packaging. `opts.palletId` forces a specific one. The palletise-over-weight threshold stays on `b2b_settings.freight_pallet_threshold_g` — it decides pallet vs cartons for the order, not which pallet. The legacy `b2b_settings.freight_pallet_*` columns are retained and `loadFreightPallets()` falls back to them when the table is empty, so a deploy landing ahead of the migration cannot leave freight unquotable.
- **A palletised order is CARTONISED FIRST, then the cartons are stacked (2026-08-27).** The original pallet path never boxed anything: it emitted `ceil(totalWeight / max_weight_g)` pallets, shared the weight evenly, and listed every SKU as flat contents. Two things followed. (1) Pallet choice was blind to dimensions — with two pallets on the same weight cap the tie-break went to the *smaller deck*, so a Hunter Mechanical cart holding 1650 mm exhausts was quoted on an 1100×1100 deck it physically overhangs. (2) Pallet count was weight-only, so 2.34 m³ of light parts was declared as one pallet (129% of that pallet's envelope) because 289 kg sat under a 400 kg cap. `packItems()` now runs the carton packer first, then stacks those cartons onto pallet slots in **layers**: `stackHeightMm()` lets the tallest remaining carton set a layer's height and fills that layer with footprints up to `AREA_FILL` 0.85 of the deck. A slot is full when the stack reaches the pallet's usable height (`max_height_mm` less `PALLET_BASE_MM` 150 for the timber) or its weight cap. **The slot count IS the pallet count** — it is no longer computed. **Declared height is the real stack, not the ceiling (Chris, 2026-08-27):** `declaredPalletHeightMm()` returns base + actual stack, rounded UP to the centimetre MachShip takes and capped at `max_height_mm` . Cube is what the carrier bills, so declaring every pallet at its maximum overcharged every order that did not fill the deck — Hunter's second pallet declared 1500 mm for a 450 mm stack. Critically, **capacity and the declared height come from the same layer model**: a slot accepted on one measure and billed on another is precisely how a pallet ends up overflowing the height it was quoted at. Orientation is chosen by `chooseOrientation()` , **natural first** (length × width down, height up — cartons have a this-way-up), tipping over only when upright will not fit; `fitsDeck()` is defined as "some orientation exists" so the two can never disagree. A carton no orientation will place comes back `loose` and ships beside the pallets as its own Carton item rather than disqualifying the pallet; selection compares pallets + loose units, tie-breaking on deck area, which is what rules the small deck out. **`loose` is a last resort, not the normal path for long items:** Hunter's 1650 × 550 exhausts lie flat on the 1800 × 1200 deck and the auto plan puts all three on pallet 1 with nothing loose — they only strand when the small deck is forced (`palletId`) or the large pallet is deactivated, which is also the one config change that would quietly turn three palletised items into three consignments. Each pallet carries its REAL weight, not an even share.

- **Freight markup is TIERED (`b2b_freight_markup_tiers`, migration 208, 2026-08-27).** One flat `freight_markup_percent` charged the same 20% on a $2,800 consignment as on a $60 one. Chris's bands: **≤$500 → 20%, $500–$1,000 → 10%, over $1,000 → 5%** (the top band left open-ended on his instruction, so nothing can fall through unmarked). The band is chosen by **what the carrier charges us ex GST**, not the sell price — also his call, and the only version that is a calculation rather than a fixed-point solve (bands read against the sell price depend on the answer, and near a boundary can have no stable one). `up_to_ex_gst` is the INCLUSIVE bound; NULL is the open-ended band and a partial unique index allows only one. `resolveMarkupPct(base, tiers, fallback)` in `lib/b2b-freight` resolves it, falling back to the legacy flat percent when the table is empty OR when bands exist but none covers the price — so an empty table prices exactly as before and a half-configured one cannot produce a zero markup. Markup is applied **per rate, not per quote**: the band depends on that carrier's price, so a $480 service and a $520 service on the same order are marked up differently, and `markup_pct` travels with the rate into `freight_chosen_quote`. **The bands are CLIFFS, not a sliding scale** — a $500.00 carrier price earns $100 of markup and a $500.01 one earns $50.01, so a dearer consignment can cost the distributor less. That is what was asked for; a smooth version would mark up each band's slice separately, the way income tax does. CRUD at `/api/b2b/admin/freight-markup-tiers` (mirrors freight-pallets, `edit:b2b_distributors`), edited by `FreightMarkupTiersManager` in Settings → Freight Pricing, where the old flat field is now labelled **Fallback markup**. Not applied to static zone rates, satchels or drop-ship freight — those carry fixed sell prices rather than a marked-up carrier quote; `dropship-calibration` still reads the flat percent for its suggestions.
- **Balancing pass (`balanceSlots`, 2026-08-27).** FFD fills pallet 1 to its height limit and dumps the rest on pallet 2, which gave Hunter 1470 mm / 258.5 kg against 600 mm / 30.5 kg. A second pass redistributes the cartons across **exactly the slots FFD decided on** — longest-processing-time first, biggest carton to the shortest stack, weight as the tie-break — and is accepted only when the SUM of stack heights does not increase (deck area is fixed, so that sum is declared cube). If any carton fails to place, or a slot would end up empty, the FFD layout stands. **It is not always an improvement and correctly declines:** 50 uniform JA-BOX-2s rebalance 42/8 → 25/25 (1050/1050 mm, 150 kg each, same cube, far better handling), but Hunter's cart is *rejected* — two of its three exhausts share one 500 mm layer, and splitting them across pallets would cost a second 500 mm layer. The lopsided split really is the cheaper one.
- **A quote prices SEVERAL packings and takes the cheapest (`packCandidates`, 2026-08-27).** Geometry cannot answer "is it cheaper to palletise some and ship the rest separately" — de-palletising cuts declared cube but multiplies per-item handling, and only the carrier knows the trade. `packCandidates()` returns up to three plans: **pallet** (all on decks), **hybrid** (own-packaging/ `packaging='pallet'` cartons on a deck, standard boxes as parcels), **cartons** (all parcels, offered only when nothing must palletise). For Hunter: 2 units / 4.47 m³, 17 units / 3.68 m³, 36 units / 2.47 m³. `getLiveQuote()` calls `/apiv2/routes` for each **in parallel**, then keeps the cheapest plan **per carrier/service** — so the distributor still chooses a carrier and each carrier is shown at its own best packing. A candidate whose routes call fails is dropped, not fatal; all failing returns the old `unavailable` reasons. An explicit `packMode` collapses this to one candidate — staff saying "cartons" must never be quietly quoted pallets. **Cost: up to 3× the MachShip calls per quote**, which is why the candidate set is fixed and small rather than a search.
- **The quoted plan is what books and prints.** Booking used to re-pack from `packMode`, which was safe while one input gave one plan — with candidates it would silently pick a different plan than the money was collected for. The winner is persisted as `freight_chosen_quote.pack_plan_units` at checkout (`checkout/start.ts`, `admin/test-order.ts`), and `orderPlanUnits(order)` is the single reader: `freight_pack_plan` (the admin's manual override) first, then the quoted units, then null = re-pack. Used by `b2b-freight-book`, `b2b-pick-list-print` and the pack-plan endpoint — all three had to gain

`freight_chosen_quote` in their SELECT, and the pick list's omission would have printed a plan the order was not shipping as. `freight_pack_plan` is deliberately NOT written: it means "a human edited this" and drives the panel's *Manual plan* badge.

- `PackedUnit.boxes` (new) is the box plan for a pallet: name, dims, weight and contents of every carton on that deck. `contents` keeps its old meaning — the flat per-SKU list — so the pick list, pack plan and admin UI were unaffected by the shape change. `parsePackPlanUnits()` round-trips `boxes` through a saved manual plan. Because pallets are now one unit each (`quantity: 1`) rather than one grouped unit, they reach the pack-plan `combine` / `setbox` paths that a `quantity > 1` guard used to block: both now refuse `itemType === 'Pallet'` explicitly, and the admin checkbox is disabled for pallets. A pallet is not a box — merging one into a carton would ship it at the envelope of the selection and throw the box plan away.
- **Bundles**: "includes" children ship inside the parent's box (affects freight cartonization).
- **Refunds**: admin order page → Refund modal → **Items mode** (pick lines + quantities; amount derived server-side from checkout-exact pricing; `refunded_qty` prevents double refunds) or amount modes (covers freight/surcharge). Mirrored to MYOB as a credit note with negative item lines (Xero path posts on B2B_SALES).
- **Drop-ship confirmations are RETRYABLE, and a dispatch notice relays tracking (migration 209, 2026-08-28).** B2B-2026-000052 (Weirys) sat unbilled and uninvoiced for two days. The watcher did its job: it matched MPI's reply on PO 00001382 and the classifier read it correctly. MYOB then rejected the PO->Bill with `FreightTaxCode is required` - fixed 30 minutes later in `fe38a23` - and the sale-invoice conversion failed with `Inventory_InsufficientStockMultipleLocation`, which is only fallout (the bill is what receives the drop-ship stock the invoice consumes). Nothing ever re-ran it: the watcher claims each message by `(mailbox, graph_message_id)` BEFORE acting and treated **any** existing row as handled, so one transient MYOB error strands an order permanently and silently. **The same trap as AP auto-entry, in a second file.** Now: a DECISION (`confirmed` / `acknowledged` / `self` / `no_match` / `not_confirmation`) is still final - re-deciding those is how you double-bill - but `error` is retried up to `MAX_CONFIRM_ATTEMPTS` (3), counted in the new `attempts` column because there is exactly one row per message here (unlike AP, where the row COUNT is the counter). The retry re-claims by UPDATE gated on `action='error'`, which gives overlapping cron runs the same mutual exclusion a fresh insert gets from the unique index. **PO matching now tolerates the supplier's own zero-padding.** The trimmed-number regex was `(?<!\d)1382(?!\d)`, which cannot match inside `JAWS-01382` because the `1` is preceded by a digit - so MPI's despatch notice logged `no_match` and its tracking number never reached anyone. It is `(?<!\d)0*1382(?!\d)` now; `21382` and `13820` still correctly do not match. **Tracking relay:** the classifier also returns `tracking_number` / `carrier`, and on a DISPATCH verdict those are written to `b2b_orders` (never overwriting an existing consignment) and sent to the distributor via the existing `sendDistributorShippedEmail`. For a drop-ship this email is the only source of consignment details - nothing goes through MachShip, because the supplier ships direct.
- **Drop-ship receiving**: supplier replies to the PO email → `b2b-dropship-confirm` cron reads it → auto-bills the PO, flips the sale order to invoice, receipts payment, relays ETA to the distributor, posts to `#jaws-orders`. Manual MYOB conversions are adopted rather than duplicated.
- **Freight & despatch (changed 2026-08-20 — read this)**: booking and despatch are now two separate steps.
  - **Book Freight** only *prepares*: it creates the MachShip consignment (left **Unmanifested**) and prints the pick slip + consignment note/labels so the order can be picked and packed. Nothing reaches the carrier at this point and no tax invoice is raised.

- **Ship Now** (`lib/b2b-ship-now.ts`) is what actually despatches: manifests the consignment, converts the MYOB Sale.Order → Sale.Invoice, receipts the payment against it, prints the A4 tax invoice, emails/pushes the distributor and stamps `shipped_at` .
- Bulk despatch is deliberately **one manifest, not N** — MachShip's manifest call also books a carrier pickup window, so manifesting ten consignments individually would raise ten pickup requests. Select the run and ship it in one action.
- This reverses the 2026-08-06 behaviour where booking manifested immediately. If nothing is reaching the carrier, the likely answer is simply that nobody has pressed Ship Now.
- Rates: per-zone + flat-rate satchels (weight-gated; satchel rows may need seeding) + drop-ship per-zone rates; calibration panel at Admin → B2B → Dropship Calibration. If a consignment goes missing at MachShip it parks as `consignment_missing` , but the poller now retries it every 6h and re-resolves the id by carrier tracking number — rebook from the order page only if that keeps failing.

- **Tune jobs**: Stripe receipt lands in the accounts inbox → hourly cron extracts it → distributor fills customer/vehicle at `/b2b/jobs` (or the login-less weekly reminder link) → nightly GH Action creates the MD customer/vehicle/note; Monday item + thank-you letter follow. One job per VIN. Staff-side management (aliases, retries, dismiss) at Admin → B2B → Tune Jobs.
- **Training**: Admin → B2B → Training → assign per distributor or per user (assignment-gated). Courses can be generated from an uploaded PDF (LLM pipeline). Edit quiz answers at `.../training/[slug]/answers` ; preview renders the real player without recording attempts.
- **Testing safely**: Admin → B2B → Test Order exercises the full real pipeline against a chosen distributor.

7.4 B2B — staff admin (`/admin/b2b/*`)

Dashboard · Catalogue (inline price/visibility edits + drawer) · Stock Order (reorder forecasting, replaces the JAWS Excel) · JAWS Stocktake (count-sheet vs MYOB on-hand, **report-only**) · Stock Wall (saved on-hand tile views; also what suppliers see) · Stock Transfer (JAWS↔VPS paired invoice+bill + MD PO) · Distributors · Suppliers (logins) · Orders · Tune Jobs · Resources (sectioned doc library, signed uploads) · Training · Settings (Stripe status, freight carriers/zones/packaging, sender address, email templates).

7.5 Accounts Payable (`/ap` , `/ap/[id]` , `/ap/statement`)

Supplier emails → Graph inbox pull → Claude extraction → triage list.

Daily SOP (Amanda): `/ap` → Pull from Inbox if needed → review each invoice (`/ap/[id]` : PDF preview, line editor with account suggestions, MD job link, supplier presets) → Approve (pushes header + lines to the right MYOB entity) or Reject. Green-triage rows support bulk approve.

Automation: `ap-auto-entry` cron (VPS, gated by `AP_AUTO_ENTRY_ENABLED`) posts clean invoices automatically and Slacks a breakdown; supplier allowlists control consolidated and pay-on-proforma handling; duplicates get a ♻️ Slack and are filed to Read/Printed; **locked-period invoices are flagged, never auto-redated**; supplier matching is suffix-blind; link-only emails (no PDF attached) are invisible to the pipeline.

A failure is no longer permanent, and no longer masquerades as "not an invoice" (2026-08-28). Two MPI Automotive invoices arrived on 26 Aug with PDFs attached, were fetched, and vanished. The log said `skipped_not_invoice` with a NULL error, no staged PDF and no Slack card - identical to how a genuine non-invoice (a T&C page, a statement, an e-ticket) is recorded. They had in fact hit the `catch` around extraction: **failing to READ a document was being filed as the document not being a bill.** The dedup guard then sealed it, because it treated ANY log row for a (message, attachment) pair as "already handled". One transient hiccup = one invoice lost forever, silently. The same mechanism killed a forwarded VistaPrint invoice via a Graph 404.

Now: extraction failures log `outcome='error'` with the thrown message, **stage the PDF so it can actually be looked at**, and are **retried up to `MAX_ERROR_ATTEMPTS` (3)**. Terminal outcomes (`posted` , `flagged` , `skipped_duplicate` , `skipped_not_invoice`) still block for good - re-deciding those is how you double-post. The `listAttachmentMeta` catch, previously a bare `continue` (silent, unlogged, retried forever invisibly), logs an error row and joins the same 3-strike flow. Only on the LAST failed attempt does a card go to Slack, so a hiccup that clears on retry 2 makes no noise. The extracted-but-empty path stays terminal and silent but now records why (`read OK but not a bill: no invoice number and no total`), so the all-nulls ambiguity that made this take an hour to diagnose cannot recur. **The dedup guard must never use `.maybeSingle()` again.** Retries mean a pair legitimately has several rows; `maybeSingle()` errors on >1, returns `data: null` , and the guard reads "never seen" - re-extracting every 15 minutes forever at LLM cost. It selects all rows and classifies them, and a failed lookup is treated as "seen" rather than "unseen" for the same reason. (Same trap as `b2b_distributor_users` .)

The MPI root cause, found on the first retry (2026-08-28): `AP extraction: model output hit the token cap and the JSON is truncated` . `runExtraction` sent `max_tokens: 4096` ; a long invoice ran past it, the JSON came back cut mid-array, and `JSON.parse` threw `Expected ',' or ']' after array element at position 9036` . Two days invisible because the caller filed the throw as "not an invoice". Cap raised to **16000** (the documented default for non-streaming, well inside every extraction model's output limit) and `stop_reason === 'max_tokens'` now throws a message that says so - truncation must announce itself rather than surfacing as a parse error that reads like a corrupt document.

A staff-photo card must not fire when the same email carried a real invoice (2026-08-28). `maybeFlagStaffPhoto` flags an unreadable IMAGE from a staff sender - a phone photo of a docket. It fired twice on the forwarded MPI email for its inline signature images while the sibling `Invoice_655307.pdf` posted $4,649.82 in the same run, saying "nothing was entered". False and alarming. It now returns silent when `ctx.invoiceSibling` is set, reusing the `hasInvoiceSibling` check already computed for the receipt-vs-invoice rule (an invoice-NAMED attachment on the same message, so it is order-independent). A genuine phone photo arrives alone and still cards. The separate "Nothing entered from a staff invoice email" summary was already correct - it is gated on `handled` , which includes `posted` .

Not every throw is a malfunction. The extractor throws `no anchors to trust extraction` as a VERDICT - it read the document fine and the document is not a bill (the signature image in a forwarded email, the T&C page, the e-ticket). The first cut of the retry work treated that as an error, which would have burned 3 attempts and paged someone over an inline `image.png` . It is matched explicitly and stays terminal + silent; everything else is a retryable error.

Amount due is not the document total (`lib/ap-amount-due-suppliers.ts` , 2026-08-28). Red Energy states the period's charges as the total, then takes a solar feed-in credit off below; the payable figure is the one after the credit, and posting the stated total books more than is owed. A pattern list like the proforma/consolidated ones (`AP_AMOUNT_DUE_SUPPLIERS` overrides). **It does not do the arithmetic** - subtracting the credit ourselves double-counts the moment the extractor nets it off itself, and there is no way to tell which it did - so it reads the PRINTED amount due / total amount payable / new balance out of the PDF text layer and uses that verbatim. Sign handling is the delicate part: a credit balance prints as `-$70.00` , `$-70.00` or `($70.00)` , and the separator is `:?` rather than a class containing a dash, because a dash there swallows the minus and turns a $70 credit into a $70 bill (caught in test). Any non-positive due is refused outright and left to a human; so is a spaced `- $285.35` , which is genuinely ambiguous. A corrected amount currently **flags for approval rather than auto-posting** - money the machine has adjusted gets looked at once; drop that `RED:amount-due-corrected` push

once real bills have proved it. `ap-statement-watch` cron reconciles statement PDFs against MYOB and emails a digest (report-only; Capricorn statements are report-only by policy). Manual statement recon UI: `/ap/statement`.

The Slack flag card is the human interface to the automation (`lib/ap-auto-entry-slack.ts`, clicks handled in `/api/slack/ask`). Each flagged invoice carries a row id in `ap_auto_entry_log` and up to four buttons: *View invoice* (7-day signed URL), ✅ *Approve & post to MYOB* (`approveAndPost` — re-extracts with the strong model, soft checks bypassed), ➕ *Create supplier* (`proposeSupplier` → threaded review → `approveCreateSupplierAndPost`), and 🔍 *Entered manually?* (`checkEnteredManually`). JAWS cards additionally get an account-choice row (`postWithAccount`).

"Entered manually?" (migration `200`, 2026-08-25) closes the automation's blind spot: staff key flagged invoices into MYOB by hand and the card stays orange forever. The check is read-only against MYOB, in three widening nets: (1) same supplier + same `SupplierInvoiceNumber` via `findExistingMyobBill` (already OCR-tolerant and amount-aware); (2) that invoice number under **any** supplier — the flag often exists *because* no card matched — adopted only when the amount agrees, so a number collision across suppliers can't link the wrong bill; (3) recent bills for the supplier at the same amount under a different number, which are **not** adopted silently but returned as threaded candidates with a 🔗 `Link bill #…` button (`linkManualBill`). A hit sets `outcome='posted'`, `entered_manually=true`, `manual_checked_by/at`, links `myob_bill_uid`, files the email away, and flips the card to ✅ *Posted manually* (`markPostedManuallyBlocks`). `entered_manually` rows are excluded from the supplier-trust counts — they're evidence a *person* handled the supplier, not the automation. Nothing is ever posted to MYOB by this path. Cards posted before the button existed can be retro-fitted in place (chat.update, so the card keeps its ts and thread) with `GET /api/ap/admin/backfill-manual-button?days=N&dry=1` (`edit:supplier_invoices`, `backfillCheckManualButtons`); it only touches still-open flags and skips cards that already carry the button, so it is safe to re-run.

> **Gotcha, fixed 2026-08-25.** Slack stores the emoji you post as its **shortcode**, so a card read back through `conversations.replies` (or arriving on a button's interaction payload) has the header `:large_orange_circle: Not auto-posted — Supplier`, *not* the 🟠 … that was sent. The first backfill run matched the literal emoji and therefore skipped all six cards it was pointed at — silently, because the skip reason was a catch-all. `OPEN_FLAG_HEADER` now accepts both forms and anchors on the words "Not auto-posted"; `markApprovedBlocks` shared the same flaw (an approved card's header would read "✅ 🟠 Not auto-posted — …") and is fixed by the same regex. Skip reasons are now specific and quote the header they saw.

Migration `200` also fixed constraint drift found while building it: `lib/ap-auto-entry.ts` has written `outcome='skipped_duplicate'` since the cross-source duplicate guard shipped, but the check constraint from migration `145` never listed the value — so **every one of those audit rows was silently rejected** (0 rows in the table) and the attachment was re-processed, and re-extracted, on the next run. The value is now allowed.

7.6 CRM (`/crm/*`)

Pipeline kanban, contacts (+timeline), campaigns (Resend), React Flow automations. Replaces Monday quote boards + ActiveCampaign + Zapier — **manual cutover steps are recorded in the project memory/notes; AC + Monday remain live in parallel until cutover**. Website leads arrive via `/api/crm/intake` (token-guarded). Three crons drive automations/campaigns/call-linkage every 5 min.

7.7 Calls (`/calls`)

CDR list with audio, transcripts, coaching analysis (per-call-type rubrics), Sentiment/Coaching/Words/Conversion tabs, live Listen/Whisper/Barge (needs `monitor:calls`), click-to-dial (`use:phone`, `NEXT_PUBLIC_CLICK_TO_DIAL=1`). **Coaching attribution: EXTENSION first since 2026-08-31, transcript as the override.** Hot-desking ended that day - every sales advisor got their own handset (Tyronne 203, Graham 4001, Kaleb 999, Dom 204, James 201), so `attributeByExtension` in `lib/calls-analysis.ts` identifies the advisor from `agent_ext` via `call_advisor_roster.extensions` and writes `effective_advisor_source = 'extension'`. A HIGH-confidence transcript self-introduction still overrides it (people answer at someone else's desk); MEDIUM no longer does, because a fixed handset beats a fuzzy one-edit name match. An extension claimed by more than one active advisor identifies nobody and is skipped - that is the hot-desk case reappearing.

Why attribution was so poor before: every advisor's roster row listed the SAME pool `{201,203,204}`, so the extension identified nobody and the transcript was the only signal. Measured over the 30 days to 2026-08-31: ext 999 52% attributed, 203 35%, 201 57%, 204 41% - roughly half of all sales calls had no advisor. The roster now carries one extension each, and `extensions.display_name` for 4001 was corrected from "Hot Desk" to Graham.

⚠ `CALLS_ADVISOR_EXT_CUTOVER` **(default** `2026-08-31` **) is load-bearing - never backfill past it.** Before that date the extensions were genuinely shared, so applying today's mapping to older calls would confidently attribute thousands to the wrong person and overwrite the transcript attributions that are correct. **The comparison is in BRISBANE time, not UTC**: `call_date` is timestamptz, and slicing the ISO string gives the UTC date, which silently excluded every call before 10am on cutover day (caught on the first backfill). Weekly coaching report posts to Slack Monday 07:00. **Negative-call (concerns) automation is fully built but switched OFF** — `CALL_CONCERNS_ENABLED=true` resumes it.

SOP if calls stop appearing: check Connections page (`freepbx_cdr_sync` freshness) → SSH to the PBX via Tailscale → check `ja-cdr-sync` systemd timer. Transcripts stale → `ja-transcribe`. Live monitor "not configured" → `ja-ami-monitor` hasn't pushed a snapshot in >20s.

7.8 Reports (`/reports/*`)

Sub-tabs: Reports · Sales Report · Sales Dashboard · Management Dashboard · Forecast · Workshop Map · Distributor Map · **Stock EOM** (§7.18, `view:stock`) · Distributors.

Builder (6 PDF report types — Workshop Performance was removed 2026-08-20; it reported over the portal workshop tables, which stay empty while MechanicDesk is the system of record) · Sales Report (live Weekly Sales Recap; **"sales" = orders taken from Monday boards + MD, not turnover**; auto-emails Ryan Mon 07:00) · Management Dashboard (JAWS weekly Excel replica from live MYOB; config-driven charts; clickable KPI history; cache warmed 05:30) · Workshop Map (nightly MD pull; `lib/workshop-map` classification is authoritative; FY picker; five tabs — Jobs Map, Quotes Map, Conversion, By State, **Vehicle Trend**: one line per vehicle series, All FY = monthly buckets, pick a month = daily buckets, measures Jobs/Quotes/Job $/Quoted $. The trend counts every invoice and quote, so its totals run higher than the map tabs, which show one dot per customer per month) · Distributor Map (quotes near each distributor vs confirmed Monday bookings). **Multi-year:** `?compare=FY,FY` returns up to three extra FY payloads alongside the primary one; only the non-map views consume them (overlaid map dots are unreadable). Vehicle Trend fetches its comparison years with one request each so a missing year degrades to this-year-only, draws them as dashed lines of the SUM of the selected vehicles (five vehicles x three years would be fifteen lines), and has a vehicle-type multi-select with an "vs others" aggregate. **Distributor jobs on Conversion:** `lib/workshop-map/distributor-jobs.ts` reads the

cached Distributor report payload and counts one job per VIN per month from `lineItems[].poNumber` — a distributor tune invoices with the car's VIN in the MYOB PO field. `seriesFromVin()` (in `lib/workshop-map/vehicle-classification.ts`) decodes the series off the VDS characters; `isVin()` rejects the 17-character non-VINs people also type in that field. FY2026: 883 VINs, 899 VIN-months, 98.9% decoded, the rest counted into OTH rather than dropped. The toggle is off by default and disabled under a state filter (a distributor invoice has no postcode). **Validation:** `scripts/check-vin-series.ts` cross-checks the decoder against the independent tuner free-text on `b2b_tune_jobs` (100% agreement on 93 cross-checkable rows) — it needs real Supabase credentials, so run it locally or in CI. Both maps export the full FY month by month as a PDF — see below · **Sales Dashboard** (daily/monthly/total sales taken, plus a quote-pipeline view — see below) · **Forecast** (admin+manager; portal rebuild of the Monday "Forecast Dashboard - Includes JAWS" — see below). Per-user report-tab allowlists control who sees what.

Just Autos on the Distributor Map (2026-08-25). `lib/distributor-map.ts` appends a synthetic entity — key `ja:home`, name "Just Autos (workshop)", located from postcode `4560` (Burnside; override with `DISTRIBUTOR_MAP_HOME_POSTCODE`) — flagged `quotesOnly: true`. It is pushed **after** the Monday booking pass and **before** the quote pass, deliberately: it must compete for quotes on the same nearest-within-radius rule, but must never bind a Monday label (adding it earlier would put it in `names` and shift the indices `matchLabel` returns).

`quotesOnly` is a contract, not a hint — every consumer must honour it: booked/capture render as `–` not `0`, it is excluded from the "All distributors" totals in both the dashboard and the PDF, and `distributorAreasForMonth` filters it out so it never reaches the weekly sales recap. Fold it into a total and the combined capture rate drops without any distributor having done worse.

Geometry worth knowing: the nearest distributor to Burnside is ~390 km (Banana Coast, Coffs Harbour), and the radius selector offers 50/100/150/250 km — so below 250 km the new entity cannot take a quote from anyone; at 250 km the areas overlap and quotes nearer the workshop move to it. The pre-existing `DISTRIBUTOR_MAP_EXCLUDE` filter (default `just autos`) still guards the `b2b_distributors` query; there is no JA row there today, so it is defensive only and unrelated to this entity.

Per-location CSV export (added 2026-08-25). Every Jobs/Quotes map popup carries a **⬇ CSV** button (`exportLoc` in `components/workshop/WorkshopMapDashboard.tsx`) that downloads all points behind that dot — the popup renders only the 40 largest. Client-side only: it serialises the already-loaded payload points, so there is no route and no gate beyond the page's own `view:reports`. Popup content is an HTML string, so the handler is attached on `popupopen` (Leaflet rebuilds the node each open), and the blob is written with a UTF-8 BOM so Excel doesn't mangle customer names.

The point date needed for it is a new `d` key (`YYYY-MM-DD`) on payload points in `lib/workshop-map/build-payload.ts` — the rep invoice's issue date for jobs, the quote date for quotes. **Payloads cached before 2026-08-25 have no `d`**, so the Date column stays blank until the nightly `md-workshop-map` pull rebuilds them (or someone hits "Pull from MechanicDesk now"); the Month column is populated either way. The `i` key was already the MechanicDesk *display* number, not the internal id — `withDisplay()` in `scripts/pull-md-workshop-map.ts` swaps it in before the payload is built.

Map PDF exports (added 2026-08-25). Both map reports have an **Export PDF** button that prints the whole FY month by month — the screen shows one month at a time and there was no way to take a year off it.

	Workshop Map	Distributor Map
Renderer	`lib/workshop-map-pdf.tsx`	`lib/distributor-map-pdf.tsx`

	Workshop Map	Distributor Map
Route	`GET /api/workshop/map/pdf?fy=&cat=&state=`	`GET /api/reports/distributor-map-pdf?fy=&radius=`
Data	the cached `md_workshop_map_cache` payload — same one the screen reads, so it returns in ~1s and never triggers an MD pull	recomputes via `computeDistributorMap` (Monday is live), ~seconds, `maxDuration` 120
Gate	`view:reports`	`view:reports`

Both follow the house `@react-pdf/renderer` style of `lib/jaws-stock-eom-pdf.tsx` (A4, Helvetica, `fixed` table headers that repeat on page breaks, `wrap={false}` rows). Neither is month-filtered by design — the month strip narrows the screen, not the export; the vehicle/state (workshop) and radius (distributor) filters *do* carry through, into the content and the filename.

Two populations in the workshop PDF, and they are not interchangeable. Counts come from `payload.conv` (every deduped record in the FY); revenue comes from `payload.*.points` (geocoded records only — an ungeocoded job carries no amount in the payload at all). Every table says which it is using and the last page prints the coverage. A "total revenue" that mixed them would be wrong, which is why they are kept apart rather than added.

`pcState()` (postcode → state) moved out of `WorkshopMapDashboard` into `lib/workshop-map/postcode-state.ts` so the dashboard and the PDF classify identically — they previously would have held separate copies.

Weekly Quotes & Jobs Map Report (`lib/workshop-map-weekly-report.ts` , cron `/api/cron/workshop-map-weekly` , Mon 07:10). Aggregates the previous 7 days of `md_quotes` / `md_invoices` by locality and vehicle group against a prior-4-week baseline, has Claude write the “what it means / where to market” read, and emails Matt (cc Ryan, Chris). `?dry=1` returns the JSON without sending; `?days=N` widens the window.

> **Gotcha, fixed 2026-08-24.** Both fact-table reads used a big `.limit()` , which PostgREST ignores — it caps responses at 1000 rows and drops the rest **silently**, off the end of the sort order. As the 5-week quote window grew past 1000 rows the newest week was the part that got truncated, so the report under-counted for weeks (222 reported vs 290 actual on 16 Aug) and then read a flat **“0 quotes issued” against 313 real quotes** on 24 Aug. Both reads now page via `selectAllRows()` ordered on the tables’ primary keys. The failure mode to remember is that it produces a *plausible smaller number*, not an error.

Sales Dashboard (`/reports/sales-dashboard` , added 2026-08-21). The portal rebuild of the Monday **"Sales Dashboard"** (2079976). `view:reports` , same as the Sales Report. **Three** views behind the one tab, each fetching only when first opened — this is also where the Monday **"Management Dashboard"** (321206) was rebuilt, per Chris 2026-08-21:

1. Management (default) — the Monday Management Dashboard, widget for widget: sales orders vs target **per month** (last 18), **per salesperson** (current month) and **per day** (last 30), plus the **Cancelled** and **Postponed** order totals.

Each chart keeps its own period because each target is stated per that period — the giveaway was that the five reps on Monday's P/P chart sum to roughly ONE month's total, so it is a monthly per-person target, not a running one.

- **Targets** resolve DB-first through `lib/integration-config`: `SALES_TARGET_PER_MONTH` / `SALES_TARGET_PER_PERSON` / `SALES_TARGET_PER_DAY`, defaulting to the values read off Monday's target lines (**$1,000,000 / $300,000 / $50,000**). Changeable without a deploy.
- **Cancelled and Postponed are whole-board GROUP totals with no date filter**, and the group is the authority, not the status column — the Postponed group holds rows whose status says "Done" or "Not Done" and they still count. Verified to the cent on 2026-08-21: the 12 rows in "Postponed Orders" sum to **$73,804.45**, exactly what the Monday widget shows. `?exceptions=1` on the API opts into this extra pull; only this view needs it. "Cancelled Orders - Previous Years" is a separate group, **excluded by default** — `SALES_EXCEPTIONS_INCLUDE_PREV_YEARS=1` folds it in.
- These exception totals are **excluded from every sales figure** — cancelled and postponed work is not revenue. They sit alongside as what was lost and what is parked.
- **"Staff Parts Owing" is NOT built.** There is no board of that name (the dashboard names 9 connected boards), so it must be a filtered view on one of them; the tile renders as "—" with a note rather than inventing a number. Reads $0 in Monday today.
- Per-salesperson excludes **Unassigned** so the rep comparison isn't distorted by rows with nobody set; those are reported on the Figures view instead.

2. Figures — what the Monday dashboard actually tracks: **daily, monthly and per-salesperson sales taken over any date range**. `lib/sales-figures-monday.ts` + `GET /api/reports/sales-figures?start=&end=&months=&days=&person=`. An explicit `start`/`end` wins over `months`; a backwards range is swapped rather than returning nothing. Built **on `fetchOrders` / `fetchDistBookings` from `lib/sales-recap-monday`** rather than re-pulling the boards, so the definition of "a sale" lives in ONE place — those already drop the dead statuses (Deleted / Canceled / Cancelled) and the Distributor-Booking groups that don't count (Booking - Pending, Postponed…). Change it there and both this and the Weekly Sales Recap follow. Daily fills every calendar day so quiet days and weekends read as gaps rather than closing up the axis; the average is per **trading day** (days with at least one sale), because dividing by calendar days quietly halves it. Best day and best month are period-wide, not limited to the daily window. Month/year-to-date return 0 when the range ends before today, rather than looking like a fault.

Salesperson attribution. The people column — "Created By" on Orders, "Person" on Distributor - Booking, both id `person` — was added to `ORD`/`DB` and to `OrderRow`/`DistRow` in `lib/sales-recap-monday` for this (additive; the Weekly Sales Recap and Distributor Map ignore it). A row can name **several** people; it counts against the **first** only, so the per-person rows still add up to the period total — counting a shared row against everyone named would inflate the total and make the table irreconcilable with the headline. Rows with nobody set group as **Unassigned** rather than being dropped. `?person=` scopes the tiles, charts and sale-type split; the per-salesperson table always covers the whole range so you can still see everyone while filtered.

3. Pipeline — the quote pipeline across the five rep Quote Channel boards. `lib/sales-dashboard-monday.ts` + `GET /api/reports/sales-dashboard?months=3|6|12|24`. Open pipeline by stage, won/lost by month, a per-rep table with win rate, and an ageing breakdown.

Sales taken ≠ turnover. Both views count orders/bookings *placed*, the same meaning the Sales Report uses; invoiced turnover is Reports → Forecast. The two will not agree and are not meant to.

Three things the Pipeline view has to get right:

- **Group ids drift per board — resolve by TITLE, never by id.** Only the five original template groups (`topics`, `group_title`, `new_group__1`, `new_group`, `new_group860`) are shared across the boards. "Lead RLMNA", "Follow up RLMNA", "On Hold" and "Not issued" were added after the fork and every board has its

own ids — Kaleb's On Hold is `group_mm12crrx`, Dom's is `group_mm12q0j0`. **The `GROUPS` map in `lib/monday-followup` holds one board's ids and is correct for the shared five only — do not reuse it for cross-board reporting.**

- **Rep comes from the board, not the Owner column.** The Owner backfill of 2026-08-20 covered the ~704 *active* quotes; historical Won/Lost items have an empty Owner, so board-as-rep is the only attribution that carries history.
- **Open is defined by exclusion** — everything that is not Won, Lost or Not issued. That way a group nobody mentioned still appears instead of vanishing: Kaleb has a "Farm Fest 2026 booked in Jobs" group, which is counted in the pipeline *and* listed separately on the page so the totals can always be reconciled to the boards.

The columns it reads *are* shared across all five boards (verified 2026-08-21): `numeric_mkzcbhz2` Quote Value (titled "Value" on James's board, same id), `date4`, `status`, `person`. The ones that drift — Distributor, Qualifying Stage, Contact Attempts, FU Stage, Quote No — are not read. Monday does the group and date filtering server-side, so the ~6,500 items across the boards cost one group-resolve plus two queries per board.

Forecast (`/reports/forecast`, added 2026-08-21). The portal rebuild of the Monday **"Forecast Dashboard - Includes JAWS"** (dashboard 349826), which sits on the Monday **"Forecasting"** board `1842188200`. `lib/forecast-monday.ts` pulls the board live on every load — 24 items, one Monday query, nothing cached or stored — and `GET /api/reports/forecast` serves month × entity × year turnover. Admin + manager only, same as the Management Dashboard.

Three things about that board that the parser has to defend against, all confirmed against live data:

- **Item names are unreliable, so the month always comes from the GROUP.** The December group contains an item titled "November Job Report - JAWS", March's is "Mach Job Report" (typo), and August's and September's are simply "JAWS". Reading the month from the title silently misfiles rows.
- **The board's "% Increase/Decrease" column is sparse and inconsistent** (populated on a handful of JAWS rows). It is carried through for reference, but every percentage the report displays is computed from the turnover figures.
- **Months after the current one hold orders already booked, not turnover earned.** They are drawn outlined rather than filled and excluded from the year-on-year headline — summing them against a full prior year would understate the business badly. Only completed months (`lastCompleteMonthIndex`) feed the totals.

Entity is `JAWS` when the item name mentions JAWS, `VPS` otherwise; blanks stay `null` rather than collapsing to `$0`, so "no data" never reads as "no turnover". The year-on-year chart deliberately shows **two** series, not three: three years of one hue fail the normal-vision colour-separation floor (ΔE 14 — indistinguishable even with full colour vision), so 2024 lives in the table beneath. Series colours are CSS variables with separately validated light and dark steps; note that CSS-var colours do not resolve in SVG presentation *attributes*, so every fill goes through `style={{}}`.

Where the reports get their data (audited 2026-08-20 when the workshop module was parked — nothing else needed reworking):

Source	Sections
MYOB (`lib/myob-reporting`)	KPI summary, P&L, top customers, receivables/payables aging, stock summary/reorder/dead, distributor ranking, pipeline, 6-month trends

Source	Sections
Monday boards	Sales funnel, rep scorecards, quote aging, monthly conversion trend, combined pipeline
MechanicDesk facts (`md_invoices` / `md_quotes`)	Workshop Map, incl. Vehicle Trend
`calls` + `call_analysis`	All six Call Analytics sections
`crm_*`	Lead pipeline, campaign performance
`b2b_*`	B2B sales

None of them read the portal's workshop tables — that was only ever Workshop Performance, now removed.

7.9 Tasks & Projects (`/tasks`, `/projects`, `/todos`)

Tasks: Monday-style board/list + React Flow automations (5-min cron). Projects: force-directed web of the six Monday "Hidden To Do" boards with comment posting back to Monday. To-Dos: manager scorecards over the same boards.

7.10 Messaging (`/messages`) & Notifications

Portal-native channels/DMs (Slack replacement for internal chat; WhatsApp/Messenger/IG phases not built). Notification bell + badges with 8 emitters; Web Push for staff and distributors (PWA installable).

PWA link capturing (2026-08-25). Two manifests are served — `public/manifest.json` for staff (`scope: /`) and `public/manifest-b2b.json` for distributors (`scope: /b2b`), chosen per route in `_document.tsx`. Both now declare `"handle_links": "preferred"` so a Chromium browser opens in-scope URLs in the installed app instead of a tab, and `"launch_handler": { "client_mode": "navigate-existing" }` so a link reuses the open window rather than stacking new ones. Both also pin `"id"` to their existing `start_url` — the implicit id *is* `start_url`, so this changes nothing for an already-installed app while stopping a future `start_url` edit from re-identifying it as a different app.

Limits worth knowing before anyone reports it as broken: **iOS has no link capturing for home-screen web apps at all** — Safari always handles the link, and only a native wrapper would change that. On **Android** the WebAPK has to refresh before capturing starts (Chrome does this in the background, typically within a day; reinstalling forces it). On **desktop** the user may get a one-time "open in app?" prompt, and the behaviour is also togglable per-app in Chrome's app settings. Note the two scopes overlap — staff `/` contains `/b2b` — so on a device with *both* apps installed a `/b2b/...` link has two candidates; that combination shouldn't occur in practice (staff device vs distributor device) but it is the reason to be careful before widening the B2B scope.

7.11 Agents (`/agents`)

Monitoring-agent inbox (`lib/agent-framework` — not `lib/agents`, which is Pipeline A's mailbox map). Agents run every 15 min via cron; findings are triaged Dismiss / Mark done.

7.12 Stocktake — Mechanics Desk (`/stocktake`)

Upload count XLSX → Run Match (dispatches GH Action; includes coverage check) → review variance → Push to MD. Counted rows are sacred (row_number bug fixed 2026-07-30). MD is single-session — a stocktake run can 401 if someone logs into MD as the same employee mid-run (known, deferred).

⚠ **System QTY: "available" is NOT on hand (fixed 2026-08-27).** `/auto_workshop/resource_search`, which the SKU match uses, returns `available` **only** — no `quantity`, no `allocated_quantity` (the worker log proves it: `quantity=undefined, available=2, allocated=undefined`). Available is on-hand MINUS allocated, so a part at zero stock with one unit allocated to a job reports **−1**. The Stock Value pass (`applyTotalQty` off `/stocks.json`, which carries **both** `quantity` and `available_quantity`) corrected most rows — but `fetchInStockUniverse` filtered to `available > 0`, so parts sitting at ZERO were never corrected and kept the search figure. On the 27 Aug count that was 326 of 358 rows corrected; `04111-11431` and `FFVDJ79-EXH-DBK-JA` were left at −1 and then **pushed into MD stocktake 50781**, after which the recheck read them straight back — which is why they carried `count_source: 'md_stocktake'` and looked like MD's own numbers. They were ours. Three changes: `totalOnHandFromStock()` / `availableFromStock()` in `lib/mechanicdesk-stocktake.ts` are now the single extractors, and the total one **deliberately excludes every available-style key** (it returns `available + allocated` where both exist, else undefined — never bare available); `fetchInStockUniverse(..., { includeAll: true })` keeps zero and negative on-hand rows so the system-QTY pass covers every counted part, with coverage filtering `> 0` in memory off the same pull rather than re-pulling; and `md_qty_provisional` flags a row whose figure came from bare `available`, cleared as soon as Stock Value supplies a total. The provisional stand-in is kept rather than blanked so a failed Stock Value pull leaves a rough baseline instead of zeroing all 358 rows.

⚠ **Corrected the next day (2026-08-28) — the 27th's fix was incomplete in two ways, and its central rule was wrong.** Chris, after running a refresh and still seeing −1: *"on hand should be 0. the -1 comes from an allocated part in the future thats not yet received."*

1. `runRefresh` **was never fixed** — only `runMatchPostPass` was. Refresh called `fetchInStockUniverse` unfiltered-flag-less, so it still skipped zero-on-hand parts. It now pulls with `includeAll: true` for the qty map and takes an in-stock **view** (`filter(u => u.available > 0)`) for the orphan question, which is genuinely about what MD still stocks. Same one-pull, two-views shape as the match pass.
2. **The recheck read-back undid everything.** `runRecheck` overwrites `md_current_qty` from the MD stocktake sheet, and the sheet held the −1 the portal itself pushed on the 27th. The morning of the 28th ran recheck → refresh (338 rows corrected) → **recheck again**, and the last one copied −1 straight back. A value can be laundered through MD and returned looking authoritative.
3. `zeroProvisionalNegatives` **is replaced by** `clampSystemQtys`**: a stocktake system QTY is never negative, whatever the source.** This SUPERSEDES the previous "a genuine negative from MD's `quantity` is real, preserve it" rule. A negative is a TIMING artefact — stock committed or sold before it arrived — not a quantity on a shelf, and you cannot count minus one of something. Leaving it also inflates the variance by the allocated amount and paints the row red for a discrepancy that does not exist. Applied as a **sweep immediately before every save**, not at each assignment, because `md_current_qty` is written from four places (SKU search, Stock Value, refresh, MD read-back) and a rule enforced in three of four is not a rule.

Deploying the fix does not clean existing data. The stored rows and MD stocktake 50781 both still hold the −1 until a recheck or refresh runs against the new code; MD's own sheet keeps its copy until re-pushed.

7.13 Stripe→MYOB (`/stripe-myob`)

Lists Stripe invoices per account label and pushes them to MYOB as Professional Invoice + Customer Payment pairs; payout reconciliation endpoints support the JAWS accounts.

7.14 Sales/analysis surfaces

`/distributors` (group-aware revenue reporting; groups managed at `/admin/groups`), `/sales`, `/stock` (⚠ no SSR gate), `/forecasting` (MD job report forecast; `/jobs` redirects here), `/vehicle-sales` (platform classification cache from VPS invoices), `/job-reports` (MD job report ingest for PO→job matching).

`/distributors` grouping — one rule for every tab (2026-08-21). Groups live in `dist_groups` / `dist_group_members` (`/admin/groups`) across two dimensions: **type** = Distributors / Sundry / Excluded, **region** = National / International.

Until this change each tab had its own idea of scope, which is why the numbers never tied up:

Tab	Was
Summary	Sectioned by dimension, Sundry separate — correct
National P/M	Genuinely National-only, filtered **server-side** (`monthlyNational` in `pages/api/distributors.ts` keys off `location === 'National'`, excluding International *and* Sundry)
National Total	Titled "National" but contained **everything**, International and Sundry included
Distributor Sales · Detailed Sales	Silently **included** Sundry
Parts : Tunes	Excluded Sundry via its own `typeOk` check

Now `sectionOfLine` in `pages/distributors.tsx` is the single definition, and a **Group by / Showing** bar under the tab strip drives every tab through `visibleLines`. Notes that matter:

- **Sundry is its own section in BOTH dimensions**, matching how the Summary has always rendered it. Otherwise a Sundry customer would be counted inside National or International and the sections would stop reconciling. Membership bears this out: all 23 Distributors carry a region, and the 15 customers with no region are all Sundry — so under region grouping there is no "Unclassified" section to explain away.
- **The monthly trend is now derived client-side** from the same `filtered` lines as everything else. The server's `monthlyNational` aggregate is hardcoded to National and could never follow the selection; it is still in the payload but deliberately unused. Fix the server aggregate too if anything else ever needs it.
- **Changing dimension resets the section to All** — a "Distributors" filter means nothing under `region`, and leaving it set would empty every tab.
- **Parts : Tunes keeps its Sundry exclusion while Showing is "All"** (Chris 2026-07-22), but honours an explicit section choice — otherwise picking Sundry would blank the tab and look broken.
- **Charts plot every customer; the totals are NUMBERS, not bars** (Chris 2026-08-21). `chartRows` carries both, flagged by `isTotal`, and every chart plots `chartRows.filter(r => !r.isTotal)` while `<ChartTotals/>` renders the group totals and grand total as a text strip beneath. Two failed attempts are worth not repeating: collapsing each group to a single bar destroyed the per-customer detail that was the whole point ("so you can see accurately what each made"), and total *bars* dwarf their own members — a group total is by definition larger than anything in it, so the axis rescales and every customer bar is squashed. In a pie a total slice is worse still: counted twice, percentages meaningless.
  - Category stacking (Tuning/Parts/Oil + custom) still applies within each customer's bar.
  - **A dashed divider separates one group's bars from the next** on both bar charts, drawn by `groupSeparatorPlugin()` — a hand-rolled Chart.js inline plugin, because the annotation plugin isn't loaded (Chart.js 4.4.1 comes from the CDN, so config-level `plugins: []` is the only hook). It reads the

`section` field on `chartRows`, so a new grouping dimension needs no chart changes. Canvas can't resolve CSS variables, hence the literal stroke colour rather than a `T` token. Bars only — a pie has no axis to divide.
  - The **grand total row on the Summary table** is built from `distSummaries`, the same rows the sections above are built from, so it always reconciles to the sum of the section totals.
  - The **split mode of the National P/M trend is one line per GROUP**, not per customer — a 47-line time series is unreadable. It previously showed only the top 8 distributors, silently dropping the rest with no total to fall back on. Flag if per-customer lines are ever actually wanted.
  - The vehicle-model chart on Distributor Sales is a different dimension and is unchanged.
- **Negative categories are real and render as negatives.** CP Performance carries Parts −$3,695.45 against Tuning $9,454.55 (total $5,759.10) from credits — the only negative in the payload as at 2026-08-21. A bar extending the other way there is correct, not a rendering fault (confirmed with Chris). Don't "fix" it by clamping to zero: that would hide a real credit and stop the chart reconciling with the table.

7.14a JA Assistant — removed from the UI (2026-08-21)

The floating AI assistant that sat bottom-right on every page is **no longer mounted**. It overlapped figures on the reports, so `<GlobalChatbot />` was taken out of `pages/_app.tsx`. Nothing else changed:

- `components/GlobalChatbot.tsx` still exists and still compiles. `ChatContextProvider` is still wrapped around the app, and the nine pages that call `useChatContext().setContext({...})` (calls, dashboard, distributors, overview, projects, reports, sales, stock, todos) are untouched — they set context that nothing now reads.
- **To bring it back, put `<GlobalChatbot />` back inside `<FeedbackProvider>` in `_app.tsx`.** That is the only change needed.
- Its API surface is now **unused but live**: `/api/chat.ts` and `/api/chat-sessions/*` (list, get, delete, send) plus their Supabase session/message tables. Left in place so the feature can be restored, and because the stored history is the user's. If the assistant is ever dropped for good, those routes, the tables and the nine `useChatContext` call sites are the cleanup.

7.15 Settings (`/settings`)

Tile launcher: General · Connections (Integrations / Health / MYOB) · Distributor Report config · VIN Codes · Users · Audit log · Profile · Claude connector (MCP tokens) · Service tokens · **Leave Notifications** (§7.17) · **Library**.

7.16 Library (`/admin/library`)

This document and the SOP, served inside the portal — readable on screen with a contents rail, and downloadable as PDF. Registry: `lib/library-docs.ts` (one row per document). Gated on `admin:settings`.

- `docs/*.md` is the **source of truth**; the reader renders it live server-side (`marked`), so editing the markdown updates the on-screen copy at the next deploy.
- `docs/*.pdf` is a **generated artifact** — regenerate with `scripts/render-doc-pdf.js` (marked + Playwright; needs `npx playwright install chromium` once) and commit it. It goes stale silently otherwise.
- ⚠ **The PDFs are deliberately not in `public/`.** This document names where every credential lives, the Supabase project id, the Tailscale addresses and the open security gaps; anything under `public/` is served to the internet to anyone holding the URL, with no sign-in. Downloads go through `/api/admin/library/[slug]`, which is auth-gated.

- ⚠ `next.config.js` → `outputFileTracingIncludes` **must list** `docs/**` **for the three Library routes.** The paths are built at runtime so Next's tracer can't see them; without it the files are pruned from the serverless bundle and 404 in production while working perfectly in dev.
- **Illustrated documents.** Screenshots live in `docs/img/` and are referenced relatively (`![alt](img/foo.png)`). The renderer writes its print page to a temp file beside the markdown and loads it as a real `file://` URL — a `setContent()` page has an `about:blank` origin and silently renders file images blank. `DOC_DATE` overrides the cover's compiled date. Portal screenshots are produced from the real components (a throwaway page under `pages/` rendering the page component with mock props, driven by Playwright) — no live data is needed and none is captured.
- ⚠ **The in-app reader does not rewrite relative image paths**, so an illustrated document renders its pictures in the PDF but shows broken images on screen. `docs/library-access-sop.md` (how to find these documents, with screenshots) is therefore **deliberately not registered** in `lib/library-docs.ts` — it is handed out as a PDF. Registering it needs an auth-gated `/api/admin/library/img/[file]` route plus a `renderer.image` rewrite in `pages/admin/library/[slug].tsx`.

SOP: see `CLAUDE.md` §1 — every change to the portal updates these documents as part of the same work.

7.17 Leave decision emails (Settings → Leave Notifications)

Staff apply for leave through the monday.com **Payroll & Leave Applications** board (`5027074711`) — a WorkForm drops the application into the *Leave Applications* group. When a manager sets the **Leave Approved** column to Approved or Denied, the portal emails the applicant. Built 2026-08-24 (migrations `197` , `198`).

- **Engine** `lib/leave-decision-emails.ts` , driven by `/api/cron/leave-decisions` every 15 minutes. Admin UI + manual run: `components/settings/LeaveNotificationsTab.tsx` → `/api/admin/leave-notifications` (`admin:settings`).
- **Why not a monday automation.** monday can only send mail through the Gmail/Outlook integration: it sends from one person's connected mailbox, dies quietly when that connection lapses, and does nothing at all when the board's Email Address column is empty — which is most rows, because managers hand-create them. The portal resolves the address itself instead.
- ⚠ **What counts as an application.** That board doubles as a daily attendance log — "*Kaleb Rowe left work at 11am today*", "*Public Holiday*", "*Easter Monday - all staff*", "*TIME OFF/ OVERTIME EXPORT*" all sit on it marked **Approved**. Emailing on "status = Approved" alone would mail people about those. The rule (Chris's call) is *an item that was in **Leave Applications** and had Approved pressed on it*, and the board's own automations make it checkable: pressing Approved moves the item to **Upcoming Leave — Approved**, Denied moves it to **Leave Denied**. So only items in `topics` , `group_mkqz6qh6` or `group_mkqzjmed` (`APPLICATION_GROUPS`) are ever acted on; anything in the payroll groups is ignored outright — no email, no log row, no HR notice.
- **Address resolution**: the board's Email Address column → `leave_staff_directory` (name-as-typed → email, editable in the portal) → unresolved. A column address whose domain is a near-miss of ours is **rejected** rather than used (a live row reads `jarred@justaustosmechanical.com.au` , which would bounce into a void with everyone believing the applicant was told). Directory matching is tiered: exact, trailing noise words stripped ("*Dom Simpson Sick*"), first + surname initial ("*Chris R*"), then a lone first name only when exactly one person has it — "*Matt*" (Huddy / Smith / Karger) stays unresolved on purpose, and a row naming several people ("*James, Kaleb, Graham, Dom and Tyronne*") refuses to match rather than mailing the first one. `scripts/check-leave-resolver.ts` exercises all of this against the real board names (`npx tsx` , no network).

- **Unresolved** items are logged `no_address`, HR is emailed **once**, and every later run retries — so adding the address to the column or the directory is all that's needed; nobody has to re-approve anything.
- **Dedupe / going live.** `leave_decision_emails` holds one row per (item, decision) and that *is* the dedupe key. On the very first run every already-decided application on the application path is written as `baseline` and **nothing is sent** — 16 rows when this shipped. Flipping an item Approved → Denied is a new decision and does email again.
- Each send also posts an update on the monday item (" Approval email sent to …"), so the audit trail lives where HR is looking.
- **Settings** resolve DB-first through `integration_settings`: `LEAVE_EMAILS_ENABLED` (kill switch) and `LEAVE_HR_EMAIL` (copied on every email, the reply-to, and where unresolved notices go — `ryan@justautosmechanical.com.au`).
- **Known gap**: a separate board automation moves approved items into the payroll groups three days before the leave starts. If an approval were pressed and that mover ran inside the same 15-minute window, the item would leave the application path before the portal saw it and no email would go out. Practically impossible (a daily automation vs a 15-minute cron) but real; the fix if it ever bites is a per-item Send button on the Leave Notifications screen.

7.18 Stock EOM — JAWS month end (`/reports/jaws-stock-eom`)

Month-end stock report for the **JAWS** company file. Built 2026-08-24 (migration `199`). Engine `lib/jaws-stock-eom.ts`; API `/api/reports/jaws-stock-eom`; cron `/api/cron/jaws-stock-eom`; **PDF export** `/api/reports/jaws-stock-eom/pdf?month=YYYY-MM` rendering `lib/jaws-stock-eom-pdf.tsx` (`@react-pdf/renderer`, house style shared with `lib/reports/pdf.tsx`).

Why it exists alongside `/stock`. `/stock` (`pages/api/inventory.ts`) already computes the *live* picture — reorder alerts, velocity, dead stock, margin, on-order — on rolling 30/90/365-day windows. What it cannot do is compare months, because **AccountRight only ever reports today's quantity**: there is no historical on-hand to query. So each run freezes its numbers into `jaws_stock_snapshots`, and that stored history is the only source of month-on-month stock movement in the business.

What the report adds beyond the live page: the month's trading in isolation (units, revenue ex-GST, COGS, margin); stock turn and days-of-inventory; ageing of held value by last-sold date; slow movers ranked by capital at risk; margin leakage (sold below cost) and cost creep (last paid >10% above average cost — a price-review list); unfilled demand (sold while nothing available, or committed beyond on-hand); overstock (>365 days cover); supplier concentration of value and reorder spend; data-integrity exceptions (negative on-hand, stock with no cost, stock with no sell price); and any JAWS stocktake completed that month (migration `141`, still report-only).

Reuse, deliberately. It calls the same `fetchInventoryItems` / `fetchSaleInvoicesWithLines` readers and the same `lineExGst` normalisation as `/stock`, so the two surfaces reconcile instead of becoming a second, subtly different truth.

Sales-history window (Chris, 2026-08-25). Every average, the months-of-cover figure and the growth read are measured over a window chosen on the report — `?from=YYYY-MM&to=YYYY-MM`, presets 3/6/12/24 months, or two month boxes. Defaults to the 12 months ending with the reported month; `MAX_HISTORY_MONTHS` caps it at 36 (trimmed from the front, keeping the end fixed) so the MYOB read stays inside the 300s budget. `resolveHistoryWindow()` clamps `to` to the reported month — a month-end report must never average in

sales it could not have known about — and is exported so the API can compare the requested window against a stored snapshot's before deciding to reuse it. **A window the snapshot wasn't built with forces a rebuild**, on the screen and on the PDF export alike; otherwise the page would silently show figures for a different period.

Per SKU: `historyUnits` / `historyRevenueEx`, `avgUnitsPerMonth` / `avgRevenuePerMonth` (÷ months of the *window*, so a month with no sale counts as a zero), `monthsCoverAtAvg` (on-hand at the average rate — steadier than `daysOfCover`, which is the last 90 days alone; both are shown, and the difference is the seasonality signal), and `growthPct`. Report-level: a `history` block with the month-by-month `series` behind the whole-business growth read, surfaced as a *Sales by month* table on screen, in the PDF and summarised in the email. `halfOverHalfGrowth()` compares the back half of the window with the front half, dropping the middle month on an odd count so the halves are equal, and returns null under 4 months — two-month halves are noise. The 13-month invoice read still happens regardless of the window, so stock turn stays comparable; a longer window simply starts the read earlier. Window, average and growth are stored per snapshot (migration `202`) — without them an old snapshot's averages can't be interpreted.

Stock position — the over/under-stocked read (Morgan's ask via Chris, 2026-08-25). Every `EomItem` now carries `monthlySeries` (units + ex-GST revenue for every month of the window), and `stockPosition(item, n)` reads the last `POSITION_MONTHS` (6) of it against on-hand: over 6 months of cover is `Overstocked`, under 1 month `Short`, no sales in the period with stock held is `No sales`. `stockPositionList` on the report ranks Short → Overstocked → No sales → OK, then by capital, capped at `LIST_CAP`. It renders as a month-by-month grid in **the month-end email** (which is where it was asked for — Morgan gets the six columns, on-hand, average and verdict without opening the portal), and on the report screen and the PDF. It answers a different question from the slow-mover list: that ranks capital at risk over the whole window, this asks whether the holding matches recent demand. The email also prints the last 6 months of the whole company file's sales with the month-on-month change.

Dropped from display, still computed (Chris, 2026-08-25): *Value and spend by supplier* and *Data to fix in MYOB* are gone from the report screen and the PDF — neither drove a decision at month end. `suppliers` and `integrity` are still built and stored on the snapshot, so reinstating either is a render away with no rebuild.

On-hand quantity now appears in most tables (same change) — top movers, margin earners, slow movers, below cost, cost creep, unfilled demand — labelled "as at" the generation date, since AccountRight only ever reports today's quantity.

PDF export (2026-08-25). *Export PDF* on the report page downloads the whole report — headline figures, ageing, every exception list and the notes — as A4. It serves the **stored snapshot**, so the PDF always matches what is on screen and returns in about a second; it only builds live when that month has never been generated. The renderer is tolerant of pre-`201` snapshots (missing `capitalAtRisk` / `slowCapital` / `analysedValue` print as `–` rather than `NaN`), so an old month exports rather than failing. The button fetches with credentials and downloads a blob rather than linking directly, so a permission or build error surfaces as a toast instead of a browser error page.

Never-sold stock is excluded from the "not moving" analysis (Chris, 2026-08-25). A SKU on this item list that has never been invoiced is almost always a kit component never sold separately — `P-INTK-PIPE` (intake pipe *only*), `17276-52010` / `96711-35053` (intake gaskets No.1 and No.2), the distributor cutting jigs — and it dominated the dead-stock figure while carrying no possible action: **$47.6k of July's $114.7k**. Ageing, dead-90, dead-180, overstock and slow movers now run over `soldEver` (held stock with a last-sold date), and ageing

shares are of `analysedValue` rather than the whole holding, so the buckets still total 100%. The excluded count and value stay in the headline (`neverSoldCount` / `neverSoldValue`) and in the notes — the capital is reported, never silently dropped.

Slow movers are a capital measure, not a silence measure (same change). The old list was simply "nothing sold in 90 days", which missed the biggest problem in the business: SKUs that sell *steadily* while carrying a year of stock. Each item now has `capitalAtRisk` = value held beyond `TARGET_COVER_DAYS` (90) of its own demand, capped at MYOB's `CurrentValue` ; because `runRatePerDay` is 0 for a dead SKU this equals the whole stock value with no special case. A SKU joins the list when nothing sold in the 90 days to month end **or** it holds over `SLOW_COVER_DAYS` (180) of cover with at least `SLOW_CAPITAL_MIN` ($2,000) past the target, and the list ranks by capital at risk. Against July's real numbers the top three — FJA300 Intake Pipe ($67.0k at risk, 378 days cover), 4" DPF Revision ($66.7k, 371 days), F33A Sump Kit ($59.8k, 268 days) — were all **invisible to the old rule**, because all three were selling. Overstock (>365 days) is the extreme end of the same list and is still shown separately. `slow_count` / `slow_capital` land in the snapshot (migration `201`).

> **Comparing months across the change.** Snapshots written before 2026-08-25 hold the old semantics — `dead_90_value` included never-sold stock — so the trend line and the month-on-month delta step down at the change point. Rebuild an old month from MYOB (Reports → Stock EOM → **Rebuild from MYOB**) to restate it on the new basis.

Reorder scope. Suggestions are drawn **only from the Stock Order sheet** (`b2b_reorder_items` , migration `114`) — the curated list of SKUs the business actually buys. MYOB's item list is far wider and includes **kit components that are never sold separately**; those sit below their alert level permanently and swamped the list on first use (Chris, 2026-08-24). `b2b_product_bundles` cannot be used to identify them — it holds one row. Off-sheet items below their alert level are counted (`reorderExcludedCount`) and the count is shown, so a SKU that *should* be ordered is still visible as a number; the fix is to add it to the Stock Order sheet. If the sheet is ever empty the report falls back to every item and says so in its notes, rather than silently reporting "nothing to buy".

⚠ **Sales figures are bounded to the reported month.** `lastSold` , the 90/365-day windows and days-since-last-sold all stop at month end, so re-running an old month gives the same answer. This was a real bug on first use: the MYOB fetch runs to *today*, so an item sold after the month closed produced a last-sold date beyond the month end and a **negative** "days since last sold" (−11 on a July report). Each item also carries `unitsSinceMonthEnd` , surfaced as a **Sold since** column on the slow-mover and never-sold tables — a slow mover that has started selling again is then obvious instead of looking dead.

⚠ **Two approximations, printed on the report itself:**

- **On-hand is "as at generation time", not the last instant of the month.** Unavoidable — see above. The cron runs early on the 2nd to keep the gap small.
- **COGS = units × current average cost.** Invoice lines carry no cost of sale and average cost drifts, so margin ranks SKUs reliably but is *not* the P&L. Don't reconcile it to the accounts.

Access. Page and API both require `view:stock` , not just `view:reports` — the report carries costs, margins and supplier pricing, so a reports-only login (e.g. marketing) is refused. The tab is admin/manager only, and it participates in the per-user report allowlist (`visible_report_tabs`).

Settings (DB-first via `integration_settings`): `JAWS_EOM_EMAIL_TO` (comma-separated; defaults to Chris + Morgan), `PORTAL_BASE_URL` for the email's deep link. A month can be rebuilt on demand from the page ("Rebuild from MYOB") or re-run for a specific month via `?month=YYYY-MM` on the cron route.

---

8. Monitoring & troubleshooting

1. **First stop**: `/admin/connections` (or Settings → Connections → Health). 21 checks across accounting / workshop / comms / crm / phone / infra, written every 5 min by the health cron. Manual force: `curl -H "Authorization: Bearer $CRON_SECRET" https://justautos.app/api/cron/health-check?force=1`.
2. **Freshness-based checks** (PBX CDR sync, transcribe, Deepgram, MD pulls) go red when data stops arriving — the fix is on the source host/worker, not the portal.
3. **Graph mailbox rows red** → renewal cron failed or the subscription died: re-run `setup-graph-subscriptions` (idempotent).
4. **MYOB rows red** → token lapsed: re-run the connect flow (Settings → MYOB Connection). Remember refresh tokens die if unused for weeks.
5. **GH Actions rows red** → check the Actions tab on the repo; MD workers upload failure artifacts and Slack on failure.
6. **A B2B order paid but no MYOB invoice** → `b2b_orders.myob_write_error`; fix cause, retry from the admin order page. Stripe webhook itself is idempotent.
7. **Prints not coming out** → the workshop PC agent: check the tray/NSSM service; it drains pending jobs on startup, so restarting it usually clears the backlog.

---

9. Known risks, debt & outstanding items (as of 2026-08-31)

Security

- Supabase `service_role` key was exposed in an April 2026 session and **has never been rotated**. Rotation must update: Vercel env, the FreePBX host workers, the workshop print agent `.env`, and GitHub Actions secrets — all hold it.
- Repo working tree contains `agents/label-print-agent/.env` and `hardware/ja-scale-node/secrets.h` — verify both are gitignored and never committed with live values.
- `NEXT_PUBLIC_GOOGLE_PLACES_API_KEY` is client-exposed — must stay referrer-restricted.
- Three pages lack SSR auth gates (`/stock`, `/imports`, `/admin/connections`) — API-level gating covers data but the pages render.

Consistency / drift

- `lib/auth.ts` role type is missing `workshop` (use `lib/permissions.ts`).
- Migrations `148` and `153` are duplicated numbers; latest applied is `214_b2b_unified_invoice_number`, so next is `215`.
- **Other selects still relying on a big `.limit()`** rather than `selectAllRows()`. Paged 2026-08-24: the weekly quotes/jobs map report and the weekly calls report (both were already truncating), and the overnight-leads store (`lib/sales-recap-leads-store.ts` — its read is unbounded, so a 3-month custom range in Reports → Sales Report was heading for ~2.6k rows). Remaining sites are safe only while their tables stay small — measured 2026-08-24, all well under the cap: `lib/crm-campaigns.ts` and `lib/reports/fetchers.ts` (×2) plus `pages/api/crm/campaigns/index.ts` (CRM tables still empty), `pages/api/imports/[id]/{run,finalize}.ts` (max 78 chunks per import),

`pages/api/notifications/summary.ts`, `pages/api/b2b/admin/stock-transfer.ts` (109 rows), `pages/api/workshop/purchase-orders/generate-low-stock.ts` (0 rows). Re-check before any of them grows.

- `bank-payments-slack` and `slack-cleanup` cron handlers exist but aren't scheduled in `vercel.json`.
- Seven overlapping base-URL env vars.
- The Library reader can't display images (no rewrite of relative `img/` paths, no auth-gated image route), so `docs/library-access-sop.md` sits in `docs/` as a PDF-only document rather than on the Library shelf — see §7.16.
- Older docs (`03`, `05`, `07`) predate the CData decommission, B2B rollout and Xero foundation, and still describe the workshop migration as active — trust this document and the code.

Operational

- MYOB and Xero OAuth tokens need periodic exercise; Xero refresh tokens are single-use (rotation is handled, but never hand-edit the row).
- **MechanicDesk's single-session limit is a standing hazard for every MD worker.** The four scheduled scrapers (`md-stock-sync`, `mechanicdesk-prepick`, `md-workshop-map`, `mechanicdesk-pull`) now share one `mechanicdesk-session` concurrency group so they can't evict each other, and the stock sync re-logs in mid-run — but a human logging into MD can still interrupt any of them, and the other three do not yet validate completeness the way `fetchAllStock` does. `mechanicdesk-stocktake` is deliberately outside the group (it must not queue behind a catalogue sync while someone is counting) and self-heals only on its push path; its match/recheck/refresh modes still fail outright. The only fix that survives a human login mid-run is a dedicated MD employee account for automation.
- `scripts/` is excluded from `tsconfig.json`, so **worker scripts are never typechecked** by `npx tsc --noEmit`. A wrong destructure of `loginToMechanicDesk` (it returns `{ client, cookies }`) compiles happily and fails at runtime as a 401 "Please login" — distinct from the eviction message "logged in from a different computer". Typecheck a changed worker explicitly.
- The FreePBX box is CentOS 7 — camera bridges pinned to Node 16; the whole host is a single point of failure for calls, coaching, and camera alerts. `sip-loss-monitor.sh` may still be running there from the phone-dropout investigation (upstream/FreedomBroadband was the cause) — clean up when resolved.
- **Missed-call notifications can still fire during a LIVE ring-group call.** Asterisk writes the ANSWERED leg's CDR row at HANGUP, while the ring-group legs that stopped ringing are written immediately - so mid-conversation the CDR holds only NO ANSWER rows, `classifyCall` correctly reads that as missed, and the notification fires while the call is being handled. When they hang up the answered leg lands, the row flips to ANSWERED and trigger 167 retracts the notification: the bell ends up right, but it appeared. Migration 210 closed the two INSERT-time suppression gaps (exact number match instead of last-9-digits; no linkedid check) which cut ~14% of them, but the in-flight case is not fixable in the database - the sync must skip a call whose channel is still up. `asterisk -rx "core show channels concise"` on the PBX gives that signal; `/opt/ja-cdr-sync/sync.js` is unversioned Node 16 on the FreePBX box, so it needs a backup and a careful edit.
- **Queued-then-parked calls report their longest single leg, not the whole conversation.** `classifyCall` picks the longest-billsec `Dial` / `Park` leg with a `dstchannel`; on a queue call the leg carrying the true total has `lastapp='Queue'` and is excluded. Two known cases: 2026-07-20 09:30 shows 14:33 on Kaleb against a real ~30:19, and 2026-08-10 14:50 shows 25:16 on Tyronne against ~32:55. Both duration *and* advisor attribution are affected, so correcting it would move historical coaching numbers — deliberately left alone.

- **MechanicDesk is staying** (decision 2026-08-20) — the replacement build is paused, so the 9 scheduled MD scrapers are a permanent production dependency rather than a temporary bridge. The portal's workshop module is built but unused; `docs/workshop_md_parity.md` keeps the cutover checklist if it is ever revived.
- **The pallet stack is a LAYER model, not a 3D placement.** Layers cannot interlock and a short carton sharing a tall carton's layer wastes the difference, so the height is an over-estimate of a perfect pack and `AREA_FILL` 0.85 per layer is a judgement, not a measurement. It errs toward more height rather than less, which is the safe direction now that the figure is what the carrier bills — but it will not spot that three 1650 mm exhausts leave awkward strips of deck unusable. Compare a printed plan against a real pallet before trusting it on an unusual order.
- **Pallet weight caps are whatever Settings says.** Both configured pallets currently read 400 kg (the retired `b2b_settings` single-pallet value was 150 kg), so the weight bound almost never binds and the cube bound does all the work. If those caps are wrong the quote is wrong — confirm them against what the carrier and the forklift will actually take.
- Parked/off: negative-call automation (`CALL_CONCERNS_ENABLED`), portal-side calls analysis cron, Places key on New Booking quick-add (key unset), first real JAWS→VPS stock transfer pending, Live Bins hardware untested in production, live call monitoring WSS keep-alive retest pending.
- **The unified B2B number is fixed at order creation, so a retried MYOB write reuses it.** If a POST created the document but the portal failed before recording its UID, the retry gets a duplicate-number rejection and the order carries a `myob_write_error` needing a human to link it. Strictly better than the old second-document-under-a-new-number behaviour, but adoption-by-Number (as `lib/b2b-myob-po.ts` already does for hand-created bills) is the real fix.
- `JAWSB2B####` **has 9999 slots** and the allocator **raises** rather than wrapping (because `lpad` truncates, which would silently mint a duplicate). At ~350 orders/year that is decades away, but widening it needs a matching rethink of the drop-ship `-n` suffix budget — 5 digits leaves only 1 character inside MYOB's 13-char cap.
- **Admin → B2B → Settings still exposes MYOB invoice number prefix / padding / next number**, which now govern only the *fallback* allocator, not new order numbers. The section is retitled "fallback only" with a description saying so, but the fields are still editable and still look authoritative.
- **The coaching section can still miss a call whose analysis lags more than 45 minutes** (the 16:30 cutoff against a 27-min p95), and a posting day skipped entirely orphans its window — the hourly cron plus Brisbane-date marker makes that rare. The cutoff is one constant (`CUTOFF_MINUTES` in `lib/calls-daily-recap.ts`) if the analyser slows down.
- MYOB→Xero migration: foundation only — adapter waves per module still to come; both entities eventually move.

---

10. Key file index

Area	Files
Auth/permissions	`lib/permissions.ts` , `lib/authServer.ts` , `lib/auth.ts` , `lib/b2bAuthServer.ts` , `lib/b2bSupplierAuth.ts` , `lib/service-auth.ts`
Credential resolver	`lib/integration-config.ts` , `pages/api/admin/integrations.ts` , `components/settings/IntegrationsTab.tsx`
Accounting	`lib/myob.ts` , `lib/myob-reporting.ts` , `lib/xero.ts` , `lib/accounting-provider.ts` , `lib/accounting/*`

Area	Files
B2B pipeline	`lib/b2b-order-pipeline.ts`, `lib/b2b-payment.ts`, `lib/b2b-myob-invoice.ts`, `lib/b2b-machship.ts`, `lib/b2b-freight-*.ts`, `lib/b2b-dropship-*.ts`, `lib/b2b-tune-jobs.ts`
AP	`lib/ap-extraction.ts`, `lib/ap-auto-entry.ts`, `lib/ap-statement-watch.ts`, `lib/ap-myob-bill.ts`
Workshop tooling	`pages/workshop/letters`, `.../prepick`, `.../purchase-orders`, `.../stocktake`, `lib/workshop-*.ts`. Paused replacement build: rest of `pages/workshop/*` + `docs/workshop_md_parity.md`
Calls	`lib/live-calls.ts`, `lib/calls-analysis.ts`, `lib/softphone.ts`, `docs/pbx_click_to_dial_worker.md`
Mail	`lib/microsoft-graph.ts`, `lib/email.ts`, `lib/agents.ts` (mailbox→owner map)
Health	`pages/api/cron/health-check.ts`, `pages/admin/connections.tsx`
Workers	`.github/workflows/*`, `scripts/*`, `agents/label-print-agent/`, `agents/ja-freightbay/`, `hardware/ja-scale-node/`
Config	`vercel.json` (crons + maxDurations), `migrations/`